



Atomix: Your Personal Lab with AI

LY Innovation-Based Major Project-B Report

Submitted in partial fulfilment of the requirements of the Degree of

Bachelor of Technology in Computer Engineering

by

Kushal D. Soni (Roll No. [INSERT])

Kartik R. Verma (Roll No. [INSERT])

Dhir M. Thakar (Roll No. [INSERT])

Supervisor

Dr. Madhura Phadke



Department of Computer Engineering

K. J. Somaiya Institute of Technology

An Autonomous Institute permanently affiliated to University of Mumbai

Ayurvihar, Sion, Mumbai-400022

2026-27

CERTIFICATE

This is to certify that the project entitled "**Atomix: Your Personal Lab with AI**" is a bonafide work of **Kushal D. Soni, Kartik R. Verma, Dhir M. Thakar** submitted to the University of Mumbai in partial fulfilment of the requirement in Major Project, for the award of the degree of "Bachelor of Technology" in "Computer Engineering".

Dr. Madhura Phadke

Project Guide

Department of Computer Engineering

Dr. Sarita Ambadekar

Head, Department of Computer
Engineering

Dr. Vivek Sunnapwar

Principal, KJSIT

Place: Sion, Mumbai-400022

Date:

PROJECT APPROVAL FOR L. Y.

This project report entitled "**Atomix: Your Personal Lab with AI**" by

Kushal D. Soni (Roll No. [INSERT])

Kartik R. Verma (Roll No. [INSERT])

Dhir M. Thakar (Roll No. [INSERT])

is an approved Last Year Innovation-Based Major Project in Computer Engineering.

Examiners:

1. _____

Name and Signature

External Examiner

2. _____

Name and Signature

Internal Examiner

Place: Sion, Mumbai-400022

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Kushal D. Soni (Roll No. [INSERT])

Kartik R. Verma (Roll No. [INSERT])

Dhir M. Thakar (Roll No. [INSERT])

Date:

ACKNOWLEDGEMENT

Before presenting our LY Major Project work entitled "**Atomix: Your Personal Lab with AI**", we would like to convey our sincere thanks to the people who guided us throughout the course of this project work.

First, we would like to express our immense gratitude towards our Project Guide **Dr. Madhura Phadke** for the constant encouragement, support, guidance and mentoring at all stages of the project and report.

We would like to express our sincere thanks to our H.O.D. **Dr. Sarita Ambadekar** for the encouragement, co-operation and suggestions during the progressing stages of the report.

We would like to express our sincere thanks to our beloved Principal **Dr. Vivek Sunnapwar** for providing various facilities to carry out this project.

Finally, we would like to thank all the teaching and non-teaching staff of the college, and our friends, for their moral support during the course of this work and for their direct and indirect involvement in the completion of this report, which made our endeavour fruitful.

Place: Sion, Mumbai-400022

Date:

ABSTRACT

Chemistry is learnt by linking what is seen in a laboratory to what happens at the molecular level and to the symbolic equation, yet students get little safe, repeatable practice and rarely receive specific explanations of their mistakes. Virtual laboratories, immersive Virtual Reality (VR), game-based learning and AI chatbots each address part of this problem, but they are usually studied in isolation, judge experiments only as fixed sequences, and offer help that is unaware of what the learner has just done. This project presents Atomix: Your Personal Lab with AI, a Unity 6 virtual chemistry laboratory that runs on Meta Quest / OpenXR headsets and, through a run-time-injected keyboard-and-mouse mode, on ordinary Windows computers. Atomix offers eight inorganic experiments selected from an in-world Reaction Book. A data-driven free-hand reaction engine judges each experiment by the order of addition, instant overdose detection, underdose after a settling period and heating procedure, using per-reaction tolerances scaled by Guided, Standard and Expert difficulty levels, and rates every success with an accuracy score and stars.

Before and after each experiment the learner can watch recorded molecular videos or a live 3D animation that shows bonds breaking and forming and electrons being transferred. A context-aware AI Lab Assistant receives the live measurements, verdict and attempt history; it answers by voice and text through the Convai service and falls back to a grounded offline knowledge base with local speech, so help remains available without internet or a microphone. A persistent experiment history, enthalpy, activation-energy, entropy and Gibbs-energy graphs, and a randomized assessment scene with eighteen task types, a coin economy and HTML/CSV reports complete the learning cycle. The implementation comprises 110 C# scripts (39,601 lines), compiles without errors and passed all automated checks executed on the shipped logic (111/111, 167/167, 226/226 and 22/22). A controlled study of learning outcomes is identified as the next step.

Keywords: virtual chemistry laboratory, virtual reality, conversational AI, game-based learning, Unity, free-hand experimentation.

CONTENTS

LIST OF FIGURES	x
LIST OF TABLES	xii
LIST OF ABBREVIATIONS	xiii
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Definition	2
1.3 Aim and Objectives	3
1.4 Organization of the Report	4
CHAPTER 2: REVIEW OF LITERATURE	5
2.1 Literature Survey	5
2.1.1 Chemical representations and the need for visualization	5
2.1.2 Virtual laboratories	5
2.1.3 Game-based learning and gamification	6
2.1.4 Virtual Reality and immersive visualization	6
2.1.5 AI-assisted learning	6
2.2 Research Gaps	8
CHAPTER 3: REQUIREMENT SPECIFICATION	10
3.1 Introduction	10
3.2 Hardware Requirements	10
3.3 Software Requirements	11
3.4 Feasibility Study	11
3.4.1 Technical Feasibility	11
3.4.2 Economic Feasibility	12
3.4.3 Operational Feasibility	12
3.5 Cost Estimation	12

3.6 Functional Requirements	13
3.7 Non-Functional Requirements	14
3.8 Project Schedule.....	15
CHAPTER 4: PROJECT ANALYSIS & DESIGN.....	16
4.1 Introduction.....	16
4.2 UML Diagrams	16
4.2.1 Use-Case Diagram	16
4.2.2 Sequence Diagrams.....	18
4.2.3 State Diagram.....	19
4.2.4 Data Flow Diagrams	20
4.2.5 Class Diagram.....	22
4.2.6 Activity Diagram	23
4.3 Scene Architecture	24
4.4 Design Decisions	25
CHAPTER 5: METHODOLOGY	26
5.1 Introduction.....	26
5.2 Development Methodology	26
5.3 System Block Diagram	26
5.4 Reaction Recipes.....	28
5.5 Free-Hand Reaction Evaluation Algorithm	28
5.6 Accuracy and Star Rating Model	30
5.7 AI Assistant: Context Building, Routing and Intent Classification.....	31
5.8 Molecular Animation Model.....	33
5.9 Thermodynamic Model for Scientific Graphs	33
5.10 Assessment and Coin Economy.....	34
5.11 Frame-Rate-Independent Interaction Smoothing.....	35
5.12 Data Persistence	35

CHAPTER 6: IMPLEMENTATION DETAILS	36
6.1 Introduction.....	36
6.2 System Implementation	36
6.2.1 Laboratory Environment and Main Menu	36
6.2.2 Reaction Book and Learn-Before-Perform	37
6.2.3 Free-Hand Experimentation and Feedback.....	38
6.2.4 Experiment History and Scientific Graphs	39
6.2.5 In-Laboratory AI Assistant	40
6.2.6 Testing Scene, Coin Economy and Reports.....	41
6.2.7 Periodic Table, Settings and Presentation.....	42
6.3 Desktop Mode Implementation.....	43
6.4 Key Scripts.....	43
6.5 VR Implementation.....	44
CHAPTER 7: RESULT ANALYSIS	45
7.1 Introduction.....	45
7.2 Performance Measures.....	45
7.2.1 Build and Verification Results.....	45
7.2.2 Content and Coverage Metrics.....	46
7.2.3 Scientific Data Used by the System.....	47
7.2.4 Requirement Traceability.....	49
7.2.5 Measurements to be Added.....	49
7.3 Screenshots	49
7.4 Comparative Analysis.....	50
7.5 Limitations	51
CHAPTER 8: CONCLUSION & FUTURE SCOPE	52
8.1 Conclusion	52
8.2 Future Scope	52

REFERENCES	54
PUBLISHED PAPERS	56
CERTIFICATES	57
PLAGIARISM REPORT	58

LIST OF FIGURES

Figure 1.1: The Atomix learning cycle - choose, learn, experiment, receive feedback, ask the AI, review, analyse and try again	2
Figure 4.1: Use-case diagram of Atomix	17
Figure 4.2: Sequence diagram - performing a free-hand experiment	18
Figure 4.3: Sequence diagram - asking the context-aware AI assistant	19
Figure 4.4: State diagram of the free-hand reaction engine.....	20
Figure 4.5: Level 0 DFD (context diagram)	20
Figure 4.6: Level 1 DFD	21
Figure 4.7: Level 2 DFD of the reaction-evaluation process.....	22
Figure 4.8: Class diagram of the core subsystems	23
Figure 4.9: Activity diagram of the user journey	24
Figure 5.1: Phase-wise development methodology	26
Figure 5.2: System block diagram of Atomix.....	27
Figure 5.3: Accepted deviation per reagent at the three difficulty levels	30
Figure 5.4: Reagent accuracy function and star bands for H ₂ SO ₄ in reaction 2	31
Figure 5.5: Dual-path routing of AI questions.....	32
Figure 5.6: Energy profiles generated from catalog data for reactions 1 and 7.....	34
Figure 6.1: Distribution of the 39,601 lines of C# code across subsystems	36
Figure 6.2: The Atomix virtual laboratory with the FeSO ₄ experiment set up and assistant guidance.....	37
Figure 6.3: Main menu.....	37
Figure 6.4: Reaction Book with LEARN / PERFORM and difficulty selection.....	38
Figure 6.5: Learning panel playing a recorded molecular video	38
Figure 6.6: Live 3D molecular animation (bond breaking, forming and electron transfer)	38
Figure 6.7: Free-hand pouring with live measurement tooltip and tracker.....	39

Figure 6.8: Chemistry-specific failure card	39
Figure 6.9: Experiment History panel.....	40
Figure 6.10: Scientific graphs panel	40
Figure 6.11: In-lab AI assistant answering a typed question	41
Figure 6.12: Testing scene setup and coin HUD	41
Figure 6.13: Report card and exported HTML test report	42
Figure 6.14: Interactive periodic table	42
Figure 6.15: Pause menu with settings, achievements and controls	43
Figure 7.1: Automated verification checks passed per suite.....	46
Figure 7.2: Content inventory of the Atomix prototype	47
Figure 7.3: Enthalpy change and activation energy of the eight reactions	48
Figure 7.4: Entropy change and Gibbs free energy at 298 K.....	48

LIST OF TABLES

Table 2.1: Summary of the literature survey	7
Table 2.2: Capability coverage of surveyed work versus the proposed system	8
Table 3.1: Hardware requirements.....	10
Table 3.2: Software requirements	11
Table 3.3: Cost estimation	13
Table 3.4: Functional requirements	13
Table 3.5: Non-functional requirements	14
Table 3.6: Project schedule	15
Table 4.1: Key use-case descriptions.....	17
Table 4.2: Scene architecture	24
Table 5.1: Reaction recipes used by the free-hand engine.....	28
Table 5.2: Engine verdicts and feedback	30
Table 5.3: Offline assistant intents and answer sources	32
Table 5.4: Coin-economy parameters	35
Table 6.1: Desktop controls	43
Table 6.2: Key scripts and responsibilities	44
Table 7.1: Build and verification summary	45
Table 7.2: Content and coverage metrics.....	46
Table 7.3: Thermodynamic data per reaction and derived Gibbs energy	47
Table 7.4: Requirement traceability.....	49
Table 7.5: Comparison with existing systems	50

LIST OF ABBREVIATIONS

Sr. No.	Abbreviation	Description
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CAVE	Cave Automatic Virtual Environment
4	CPK	Corey-Pauling-Koltun (atom colouring convention)
5	CSV	Comma-Separated Values
6	DFD	Data Flow Diagram
7	FR	Functional Requirement
8	GBL	Game-Based Learning
9	GPU	Graphics Processing Unit
10	HMD	Head-Mounted Display
11	HTML	HyperText Markup Language
12	HUD	Heads-Up Display
13	INR	Indian Rupee
14	JSON	JavaScript Object Notation
15	KB	Keyboard (in figures: KB + mouse) / Knowledge Base (Offline KB)
16	LOC	Lines of Code
17	MP4	MPEG-4 Part 14 video format
18	RAM	Random Access Memory
19	RCT	Randomized Controlled Trial
20	REST	Representational State Transfer
21	SDK	Software Development Kit
22	SO	ScriptableObject (Unity data asset)
23	SoC	System on Chip
24	SUS	System Usability Scale
25	TTS	Text-to-Speech
26	UI	User Interface
27	UML	Unified Modelling Language
28	VFX	Visual Effects
29	VR	Virtual Reality
30	XR	Extended Reality

CHAPTER 1: INTRODUCTION

1.1 Introduction

Chemistry is learnt at three linked levels: the macroscopic level of what can be seen in a test tube, the sub-microscopic level of atoms, ions and bonds, and the symbolic level of formulas and equations [9]. Laboratory work is where the first level is experienced, yet the second level remains invisible and the third level is usually met only on paper, which is why learners often find it hard to connect an observation such as fizzing or a colour change to the reaction that causes it [10]. At the same time, practical laboratory time is limited by the cost of reagents, by safety rules that forbid reactive metals or toxic gases in a classroom, and by the availability of apparatus and supervision. Educational technology has responded with virtual laboratories [1], [12], game-based learning [2], [3], [11], immersive Virtual Reality (VR) [4]-[6], [13] and Artificial Intelligence (AI) tutors and chatbots [7], [14]. This project, **Atomix: Your Personal Lab with AI**, belongs to this domain of technology-enhanced chemistry education. Atomix is a Unity 6 virtual chemistry laboratory that runs on Meta Quest / OpenXR headsets and, without a headset, on a Windows desktop with keyboard and mouse. Inside one application the learner chooses one of eight inorganic reactions, watches a molecular-level explanation, performs the experiment free-hand with measured quantities, receives a chemistry-specific verdict, asks a context-aware AI Lab Assistant why the result occurred, reviews a persistent history of attempts, studies the thermodynamics of the reaction through scientific graphs, and finally takes a randomized, timed assessment. Figure 1.1 summarises this learning cycle, which is the organising idea of the whole system.

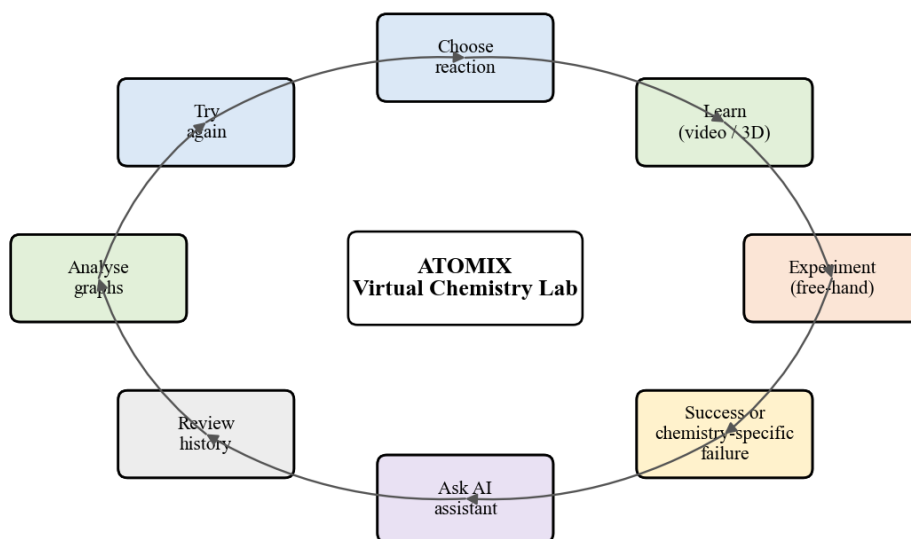


Figure 1.1: The Atomix learning cycle - choose, learn, experiment, receive feedback, ask the AI, review, analyse and try again

The cycle in Figure 1.1 deliberately treats failure as a normal and informative step. A student who pours too much sulfuric acid onto copper(II) oxide is not simply told "wrong"; the laboratory explains which reagent was out of range, by how much, and why that matters chemically, and the AI assistant can then be asked about the specific attempt. The rest of this report describes how this behaviour was specified, designed, implemented and verified.

1.2 Problem Definition

Secondary-school and first-year undergraduate chemistry students rarely get enough unsupervised, repeatable practice with real reactions, and when they do practise, they seldom receive immediate, specific explanations of their mistakes. Existing digital alternatives address the problem only partially. Virtual laboratories reproduce procedures but are usually desktop-only and judge experiments as a fixed sequence of clicks [1], [12]; VR applications offer immersion and molecular visualization but rarely assess the quantities a learner actually used [4], [5], [13]; game-based tools improve motivation but focus on a narrow concept [2], [3]; and AI chatbots answer questions without knowing what the student has just done in the laboratory [7], [14]. Moreover, most cloud-AI tutors stop working entirely when there is no internet connection, no microphone, or an expired service quota.

The problem addressed by this project is therefore: **to design and implement a single, accessible virtual chemistry laboratory in which learners can perform quantity-aware experiments in VR or on a desktop, see the molecular explanation of each reaction, receive chemistry-specific feedback on their own mistakes from an AI assistant that is aware of the live experiment and still works offline, track and review their attempts, and be assessed through randomized practical and theoretical tasks.**

1.3 Aim and Objectives

Aim. To build Atomix, an AI-assisted virtual chemistry laboratory that unifies hands-on experimentation, molecular-level visualization, context-aware conversational help, progress tracking and assessment in one Unity application that runs both in VR and on a desktop computer.

Objectives. The project objectives were defined as follows:

1. To model an interactive 3D chemistry laboratory with grabbable glassware, reagents, a Bunsen burner and a world-space Reaction Book offering eight inorganic reactions across decomposition, displacement, neutralization, redox, precipitation and combustion categories.
2. To support two input modalities from one code base: XR interaction on Meta Quest / OpenXR and a runtime-injected keyboard-and-mouse desktop mode.
3. To replace fixed-sequence triggers with a reusable free-hand reaction engine that validates reagent order, overdose, underdose (after the mixture settles) and heating duration against data-driven recipes with configurable tolerances and difficulty levels.
4. To provide a learn-before-perform workflow with recorded molecular videos and a live, code-authored 3D ball-and-stick animation for all eight reactions showing bond breaking, bond forming and electron transfer.
5. To integrate an in-laboratory AI assistant that receives the live experiment context, answers by voice and text through the Convai service, and falls back to an offline, grounded chemistry knowledge base with Windows speech synthesis.

6. To persist a detailed experiment history, compute an accuracy score and star rating for every attempt, and present energy-profile, enthalpy and entropy graphs with Gibbs free-energy reasoning for each reaction.
7. To build a randomized assessment scene with eight practical and ten theory tasks, a persistent coin economy, and an exportable HTML/CSV test report.
8. To verify the implementation through compilation against the real Unity assemblies and automated checks that execute the shipped logic.

1.4 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 reviews the literature on virtual laboratories, game-based learning, immersive VR, molecular visualization and AI-assisted learning, summarises it in a comparative table and identifies the research gaps that motivate Atomix. Chapter 3 specifies the hardware and software requirements, the functional and non-functional requirements, the feasibility study and the cost estimate. Chapter 4 presents the analysis and design of the system through use-case, sequence, state, data-flow, class and activity diagrams. Chapter 5 explains the methodology: the layered system architecture, the free-hand reaction-evaluation algorithm with its mathematical formulation, the scoring model, the AI routing logic, the thermodynamic model behind the scientific graphs and the assessment economy. Chapter 6 describes the implementation module by module with screenshots. Chapter 7 analyses the results, including verification statistics, the scientific data used by the system and a comparison with the systems surveyed in Chapter 2. Chapter 8 concludes the report and outlines future work. References, published papers, certificates and the plagiarism report follow.

CHAPTER 2: REVIEW OF LITERATURE

2.1 Literature Survey

Search strategy. We searched IEEE Xplore, ScienceDirect, the Journal of Chemical Education and Google Scholar for the period 2000-2024 using combinations of the keywords "virtual chemistry laboratory", "virtual reality chemistry education", "immersive molecular visualization", "game-based learning chemistry", "gamification laboratory", "AI chatbot education" and "educational game evaluation". Titles and abstracts were screened for relevance to (i) laboratory simulation, (ii) immersive or game-based chemistry learning, (iii) visualization of chemical representations, or (iv) conversational AI support for learners. Fourteen works were retained: eight peer-reviewed conference papers, three journal articles, one systematic literature review on evaluation, one edited book on multiple representations and one technology-support study. They are discussed thematically below and summarised in Table 2.1.

2.1.1 Chemical representations and the need for visualization

Gilbert and Treagust [9] consolidate research showing that chemistry understanding depends on moving between the macroscopic, sub-microscopic and symbolic levels, and that learners struggle precisely at the transitions between them. Wu, Krajcik and Soloway [10] studied a computer-based visualization tool that let students build and convert between molecular representations and reported that technology can scaffold the development of representational competence. Both works motivate the inclusion of molecular-level explanation alongside the macroscopic experiment, which in Atomix takes the form of molecular videos and a live 3D ball-and-stick animation tied to each reaction.

2.1.2 Virtual laboratories

Ramos et al. [1] examined how hands-on and virtual laboratories can be integrated pedagogically in undergraduate chemistry and argued that virtual laboratories are most valuable as a complement that allows repetition and preparation rather than as a replacement. Ali and Ullah [12] reviewed and compared existing virtual chemistry laboratories with respect to their features, interactivity and educational use, finding wide variation in realism, feedback and assessment support. Together they show that a virtual laboratory should allow repeated, safe experimentation and should give

meaningful feedback, a requirement that Atomix addresses with its free-hand engine and failure explanations.

2.1.3 Game-based learning and gamification

Agarwal and Saha [2] designed a puzzle-based game, "Atoms to Molecule", in which learners assemble molecules from atoms, and reported that the game format supported engagement with bonding concepts. Sanchez-Lopez et al. [3] gamified chemistry laboratory teaching through the IQ-Mobile Race activity and observed improved motivation, communication and collaboration. Byusa, Kampire and Mwesigye [11] systematically reviewed game-based learning in chemistry and concluded that it generally improves motivation and conceptual understanding, while stressing that the effect depends on the game mechanics and on clear learning goals. Kurniawan et al. [8] reviewed tools for evaluating educational-game quality and emphasised that games must be assessed for usability and educational value, not only as software. These works informed the assessment scene, the coin economy and the decision to make the mechanics themselves chemical (measuring, ordering, heating) rather than external rewards.

2.1.4 Virtual Reality and immersive visualization

Bell and Fogler [4] were among the first to apply VR to chemical-engineering education, using virtual laboratory and plant environments to visualise processes that cannot easily be shown in a classroom. Luo et al. [5] built an interactive molecular-visualization system for a multi-projection immersive environment and showed that immersive 3D presentation helps learners understand molecular structure. Alvarez Ramirez and Bueno [13] developed a VR application for learning organic chemistry. Xie, Li and Tang [6] carried out a randomized controlled pilot study of immersive VR for science education and cautioned that immersion should be judged by its measured effect on learning outcomes rather than assumed to be beneficial. These studies justify VR as the primary modality while also warning against relying on immersion alone, which is why Atomix couples immersion with feedback, explanation and assessment and keeps a desktop path for learners without headsets.

2.1.5 AI-assisted learning

Sandu and Gide [7] investigated the adoption of AI chatbots for enhancing the student learning experience in Indian higher education and found positive attitudes towards

chatbot support. Atmosukarto et al. [14] added an AI chatbot to an adaptive online chemistry course so that students could obtain answers outside class hours. Neither system, however, has access to what the student is doing in a practical activity at that moment. Atomix extends this line of work by injecting live experiment context (reaction, measured quantities, targets, verdict and attempt history) into every question and by providing an offline fallback so that help is never entirely unavailable.

Table 2.1 consolidates the surveyed works. Because several of the references are design papers or reviews rather than controlled experiments, the "Dataset / Participants" column records the kind of evidence each work rests on.

Table 2.1: Summary of the literature survey

Ref.	Author / Year	Method	Dataset / Participants	Results	Limitations
[1]	Ramos et al., 2016	Pedagogical integration of hands-on and virtual labs	Undergraduate chemistry course context	Virtual labs best used to complement and prepare for hands-on work	Desktop simulation; no immersive or AI support
[2]	Agarwal & Saha, 2011	Puzzle-based game (Atoms to Molecule)	Learners playing the game	Game format supported engagement with bonding concepts	Single concept; no laboratory procedure
[3]	Sanchez-Lopez et al., 2023	Gamified lab activity (IQ-Mobile Race)	Chemistry laboratory class	Improved motivation, communication, collaboration	Physical activity; not a simulation
[4]	Bell & Fogler, 2004	VR modules for chemical-engineering education	Engineering students / VR modules	VR visualises processes not feasible in class	Early hardware; no quantity-level assessment
[5]	Luo et al., 2004	Immersive multi-projection molecular visualization	System demonstration	3D immersion aids understanding of molecular structure	Expensive projection setup; structure only
[6]	Xie, Li & Tang, 2023	Randomized controlled pilot study of immersive VR	Science learners (pilot RCT)	Immersion must be judged by learning outcomes	Pilot scale; VR content only
[7]	Sandu & Gide, 2019	Survey of AI-chatbot adoption	Indian higher-education students	Positive perception of chatbot support	Generic chatbot; no task context

Ref.	Author / Year	Method	Dataset / Participants	Results	Limitations
[8]	Kurniawan et al., 2024	Systematic review of game-evaluation tools	Published evaluation studies	Need for structured usability and educational evaluation	Review only; no system
[9]	Gilbert & Treagust, 2009	Edited synthesis on multiple representations	Body of chemistry-education research	Macro / sub-micro / symbolic levels must be linked	Theoretical; no tool
[10]	Wu, Krajcik & Soloway, 2000	Computer-based visualization tool for representations	High-school chemistry classroom	Technology scaffolds representational understanding	2D desktop tool; no experimentation
[11]	Byusa et al., 2022	Systematic review of game-based learning in chemistry	Published GBL studies	GBL improves motivation and understanding	Effects depend on mechanics; heterogeneous studies
[12]	Ali & Ullah, 2020	Review and comparison of virtual chemistry labs	Existing virtual-lab products	Large variation in realism, feedback, assessment	Comparative review; no new system
[13]	Alvarez Ramirez & Bueno, 2020	VR application for organic chemistry	Prototype application	VR feasible for chemistry learning	Organic topics; no AI or assessment
[14]	Atmosukarto et al., 2021	AI chatbot in adaptive online chemistry course	Online course students	Chatbot extends support beyond class hours	Course-level; unaware of lab activity; online only

2.2 Research Gaps

The survey reveals that each strand of work solves one part of the problem in isolation. Table 2.2 maps the surveyed systems onto the capabilities that a complete practice laboratory needs; it is derived from the descriptions in the cited works.

Table 2.2: Capability coverage of surveyed work versus the proposed system

Capability	[1],[12]	[2],[3]	[4],[5],[13]	[6]	[7],[14]	Atomix
3D virtual laboratory procedure	Yes	-	Partial	Partial	-	Yes
Immersive VR	-	-	Yes	Yes	-	Yes
Desktop access without headset	Yes	Yes	-	-	Yes	Yes
Quantity / order-aware validation	-	-	-	-	-	Yes
Molecular-level visualization	-	Partial	Yes	-	-	Yes

Capability	[1],[12]	[2],[3]	[4],[5],[13]	[6]	[7],[14]	Atomix
Conversational AI help	-	-	-	-	Yes	Yes
AI aware of live experiment state	-	-	-	-	-	Yes
Works offline	Yes	Yes	Yes	Yes	-	Yes
Assessment and attempt history	Partial	Partial	-	Yes	-	Yes

From Table 2.2 the following research gaps are identified:

1. **Fragmentation.** Virtual laboratories, immersive VR, molecular visualization, game mechanics and AI tutors are studied separately; no surveyed system combines them in one learning flow.
2. **Shallow validation.** Simulations usually trigger a reaction when the right objects touch. They do not judge how much reagent was used, in which order, or for how long a sample was heated, and therefore cannot explain realistic mistakes.
3. **Context-blind AI.** Chatbots answer general questions but do not know the learner's current reaction, the amounts poured or why the last attempt failed, and they become unavailable without internet access.
4. **Access barrier.** Immersive systems depend on headsets or projection rooms, excluding learners who only have a laptop.
5. **Weak link between practice and theory.** Few systems connect a successful experiment to its energetics (enthalpy, activation energy, entropy, Gibbs free energy) or keep a reviewable history of attempts.
6. **Evaluation.** As [6] and [8] note, the educational effect of such systems must be measured; this remains an open item for Atomix as well and is listed as future work.

Atomix is designed to close gaps 1 to 5 within a single Unity application, and the evaluation gap (6) is addressed in this report at the level of functional verification, with a learning-outcome study planned as future work.

CHAPTER 3: REQUIREMENT SPECIFICATION

3.1 Introduction

This chapter specifies what Atomix must provide and what it needs in order to run. Requirements were derived from the problem definition in Section 1.2, from the gaps in Section 2.2 and from repeated play-testing during development. They are grouped into hardware requirements, software requirements, functional requirements and non-functional requirements, followed by a feasibility study, a cost estimate and the project schedule. Where a value is an estimate rather than a measured fact it is marked [ASSUMED].

3.2 Hardware Requirements

Atomix has two run-time targets and one development target. Table 3.1 lists the hardware for each. The desktop target is intentionally modest so that the laboratory remains usable on typical college laptops, while the VR target requires a standalone or PC-tethered OpenXR headset.

Table 3.1: Hardware requirements

Component	Development workstation	Desktop run-time (Windows)	VR run-time
Processor	Quad-core x64, 3.0 GHz or better [ASSUMED]	x64 dual/quad-core [ASSUMED]	Meta Quest on-board SoC, or PC as left
Memory	16 GB RAM [ASSUMED]	8 GB RAM [ASSUMED]	Headset built-in
Graphics	DirectX 11/12 GPU, VR-capable [ASSUMED]	DirectX 11 GPU	Headset built-in
Storage	50 GB free (project, Library, builds) [ASSUMED]	2-3 GB free [ASSUMED]	2-3 GB free [ASSUMED]
Display / HMD	Monitor + Meta Quest for testing	Monitor (any resolution)	Meta Quest / OpenXR headset
Input	Keyboard, mouse, Quest controllers	Keyboard and mouse	Quest controllers / hands
Audio	Speakers/headset, microphone	Speakers; microphone optional	Headset audio and microphone
Network	Internet for packages and Convai	Optional (cloud AI only)	Optional (cloud AI only)

A microphone is optional because questions can be typed, and the network is optional because the assistant falls back to its offline knowledge base. This was a deliberate requirement: the original assistant failed completely without internet or a microphone.

3.3 Software Requirements

Table 3.2 lists the software stack. All tools used during development are free for academic or personal use.

Table 3.2: Software requirements

Software	Version / detail	Purpose
Unity Editor	6000.3.7f1 (Unity 6), Built-in Render Pipeline	Game engine, scene authoring, builds [15]
C# / .NET (Unity scripting)	Unity-bundled	All run-time logic (110 scripts)
OpenXR plug-in, Oculus XR	Unity packages	Headset runtime abstraction [16]
XR Interaction Toolkit	Unity package	Grabbing, sockets, locomotion, world-space UI [17]
Unity Input System	Unity package	Unified keyboard, mouse and controller input
TextMeshPro	Unity package	World-space and HUD text
Unity VideoPlayer	Built in	Recorded molecular videos (MP4)
Convai REST API	character/getResponse endpoint	Cloud conversational AI and voice [18]
Windows speech synthesis	Windows 10/11	Offline spoken replies
Android Build Support / Meta Quest	Unity module	Quest (Android) builds
Visual Studio / VS Code	Any recent	Script editing and debugging
Git and GitHub	Any recent	Version control and team collaboration
Operating system	Windows 10/11 (64-bit)	Development and desktop run-time

3.4 Feasibility Study

3.4.1 Technical Feasibility

Atomix is technically feasible because every capability it needs is provided by mature, documented components. Unity 6 supplies rendering, physics, audio, video playback and cross-platform builds; OpenXR and the XR Interaction Toolkit provide device-independent headset support; and the Convai REST endpoint exposes conversational

AI and speech through a single HTTP request, which avoided the heavier SDK used in the first prototype. The team had prior experience with C# and Unity. The riskiest technical element, liquid pouring, was handled without a fluid simulation: pour volume is integrated from tilt angle and time, which is cheap enough for a standalone headset. Feasibility was confirmed in practice: the final code base compiles against the real Unity 6000.3.7f1 assemblies with zero errors.

3.4.2 Economic Feasibility

The software cost of the project is effectively zero: Unity Personal, the XR packages, Visual Studio Community, Git and the Convai free tier are free for this use, and the 3D laboratory assets were obtained from free sources. The only significant expense is hardware, chiefly one VR headset for testing, and the desktop mode means that deployment to students does not require headsets at all. The cost estimate in Section 3.5 shows that the project is within the means of a student team and a college department. Compared with recurring reagent, glassware and waste-disposal costs of physical practice sessions, a virtual laboratory that can be reused indefinitely is economical.

3.4.3 Operational Feasibility

Students already use games and laptops, so the interaction model (walk with W/A/S/D, look with the mouse, click to grab) needs little training, and a first-run onboarding card and a live objective line explain what to do. Teachers can use Atomix for pre-laboratory preparation, for remedial practice and for quick formative tests whose results are exported as HTML and CSV. Safety-critical reactions such as sodium or potassium with water, which cannot be performed freely by students in a real laboratory, can be practised without risk. Accessibility settings (motion-sickness-safe camera defaults, audio buses, typed questions) widen the range of users who can operate the system comfortably.

3.5 Cost Estimation

Table 3.3 gives an indicative cost estimate. Prices are approximate Indian market prices at the time of writing and must be verified before inclusion in any budget; they are marked [ASSUMED].

Table 3.3: Cost estimation

Item	Quantity	Approx. cost (INR)	Remarks
Development laptops / PCs	3	0	Students' existing machines
Meta Quest 3S headset (VR testing)	1	30,000 [ASSUMED]	Optional for deployment; desktop mode needs none
Unity 6 Personal	3 seats	0	Free licence under revenue threshold
Visual Studio Community / VS Code	3	0	Free
Convai API (free tier)	1 account	0	Offline fallback when quota expires
3D models, textures, sounds	-	0	Free assets; audio cues synthesized in code
GitHub repository	1	0	Free
Internet and miscellaneous	-	2,000 [ASSUMED]	Package downloads, cloud AI use
Total (indicative)		32,000 [ASSUMED]	Dominated by the optional headset

3.6 Functional Requirements

The functional requirements are listed in Table 3.4. Each is traced to its implementation and verification in Chapter 7 (Table 7.4).

Table 3.4: Functional requirements

ID	Requirement
FR1	The system shall present a 3D chemistry laboratory with grabbable glassware, reagents and a Bunsen burner.
FR2	The user shall select one of eight reactions from a world-space Reaction Book or with keys 1-8.
FR3	The system shall offer LEARN (recorded video or live 3D molecular view) and PERFORM for each reaction.
FR4	The system shall measure poured volumes, solid additions and heating time live and display them.
FR5	The system shall judge each experiment for wrong order, overdose, underdose and procedure errors against a recipe and tolerance.
FR6	The system shall display a chemistry-specific explanation of every failure and allow a bench reset (F5).
FR7	The AI assistant shall accept voice (push-to-talk) and typed questions and reply in voice and text.
FR8	The AI assistant shall receive the live experiment context and shall answer offline when the cloud is unavailable.

ID	Requirement
FR9	The system shall record every attempt (quantities, steps, outcome, AI Q&A) and persist up to 300 attempts.
FR10	The system shall score each successful attempt (accuracy %, 1-3 stars) and offer Guided, Standard and Expert levels.
FR11	The system shall show energy-profile, enthalpy and entropy graphs and let the user ask the AI to explain them.
FR12	The testing scene shall generate randomized practical and theory tasks with a timer and a chosen task count.
FR13	The testing scene shall maintain a persistent coin wallet with hints, extra time and skip purchases and ranks.
FR14	The system shall export a test report in HTML and CSV.
FR15	The application shall run in VR and, without a headset, with keyboard and mouse.

3.7 Non-Functional Requirements

Table 3.5 lists the non-functional requirements, which constrain how the functional requirements are delivered, particularly on a standalone headset and on machines without internet access.

Table 3.5: Non-functional requirements

Category	Requirement
Performance	Interactive frame rates on a standalone headset; no fluid simulation; asynchronous scene loading behind a loading screen.
Availability	Core learning features must work without internet, microphone or AI credentials.
Usability	Desktop controls discoverable via help overlay and onboarding; panels open at a readable distance; typed input gated so letters do not move the player.
Comfort / accessibility	Walking camera motion off by default (motion sickness); camera shake scalable to zero; separate audio buses; room tone off by default.
Maintainability	Reaction recipes as data (ScriptableObject assets); single shared engine for all eight reactions; frame-rate-independent smoothing.
Portability	Same scenes for Meta Quest/OpenXR and Windows; desktop mode injected at run time without editing scenes.
Data integrity	History stored as JSON with a 300-attempt cap; coins and settings in PlayerPrefs; reports written to Desktop, then Documents, then persistentDataPath.
Security	API credentials kept in a settings asset outside code; must be rotated and kept out of version control.

3.8 Project Schedule

The project was developed between April and October 2026, as reflected in the repository history (first commits in early April 2026, final commit on 1 October 2026). Table 3.6 shows the phases; the month boundaries of individual phases are approximate [ASSUMED].

Table 3.6: Project schedule

Phase	Period (2026)	Main outcomes
Requirement analysis and literature survey	April	Problem definition, reaction list, survey, research paper draft
Laboratory modelling and VR interaction	April - May	3D lab, eight reaction scripts, XR rig, Reaction Book
Desktop mode and AI assistant	May - June	DesktopBootstrap, first-person controller, lab assistant
Free-hand engine and history	July - August	Quantity-aware engine, history UI, scientific graphs
Molecular animation and offline AI	September	Live 3D molecular view, ChemistryKnowledgeBase, typed input
Assessment economy, polish and verification	September - October	Coins, reports, audio, post-processing, automated checks
Report and paper preparation	October	This report, IEEE paper

CHAPTER 4: PROJECT ANALYSIS & DESIGN

4.1 Introduction

This chapter analyses the system from the perspectives of its users, its run-time behaviour and its data. The Unified Modelling Language (UML) is used for the use-case, sequence, state, class and activity views, and Data Flow Diagrams (DFDs) at levels 0, 1 and 2 describe how information moves between processes and stores. All diagrams were drawn from the implemented code, so the class and method names in them correspond to real scripts in the project.

4.2 UML Diagrams

4.2.1 Use-Case Diagram

Figure 4.1 shows the use-case diagram. The primary actor is the **Student**, who uses all eleven use cases: selecting a reaction, learning through a video or the 3D view, performing the free-hand experiment, receiving feedback, asking the AI assistant, reviewing history, viewing graphs, taking a randomized test, spending coins, downloading a test report and configuring settings and difficulty. Two secondary actors appear. The **Convai cloud / offline knowledge base** actor answers assistant questions; it is drawn as an external actor because the cloud service is outside the system boundary, and the offline knowledge base is its substitute. The **Instructor** consumes the exported test reports and may review a student's history on the same machine.

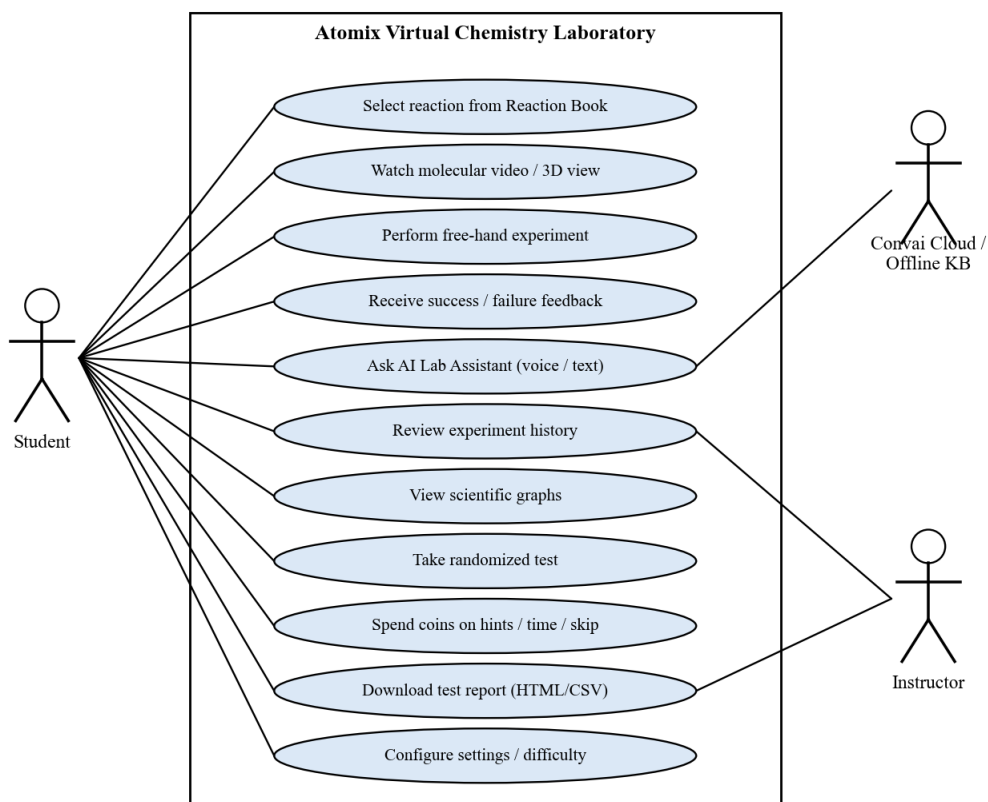


Figure 4.1: Use-case diagram of Atomix

Table 4.1 describes the three most important use cases in more detail, including their preconditions and alternative flows.

Table 4.1: Key use-case descriptions

Use case	Precondition	Main flow	Alternative flow
Perform free-hand experiment	Reaction selected in the book	Grab vessel, tilt to pour or add solid, heat if required; engine shows live quantities; mixture settles; Success	Wrong order, overdose, underdose or procedure error ends in a failure card; F5 resets the bench
Ask AI Lab Assistant	In LabScene or LabAssistantScene	Hold V or press Enter, ask; context attached; reply spoken and shown; Q&A logged	No network or credentials: offline knowledge base answers with Windows speech
Take randomized test	TestingPhaseLab loaded	Choose task count (3/5/8/10/all); timer starts; complete tasks; coins awarded; report card	Buy hint, extra time or skip with coins; time-out records the task as missed

4.2.2 Sequence Diagrams

Figure 4.2 shows the sequence of messages when a student performs an experiment. Selecting a page in the Reaction Book calls `ControlReactions.SelectReaction(n)`, which activates only the apparatus for reaction *n* and configures the shared `FreeHandReactionEngine` from the reaction's `ReactionDefinition` asset. Each pour or solid addition is reported to the engine through `AddQuantity`. The engine's `Evaluate` method runs every frame; it returns `InProgress` until it reaches a verdict, after which the reaction script plays the corresponding visual effect, sound and equation (or the failure reason) and the attempt is written to the history manager, which serialises it to JSON.

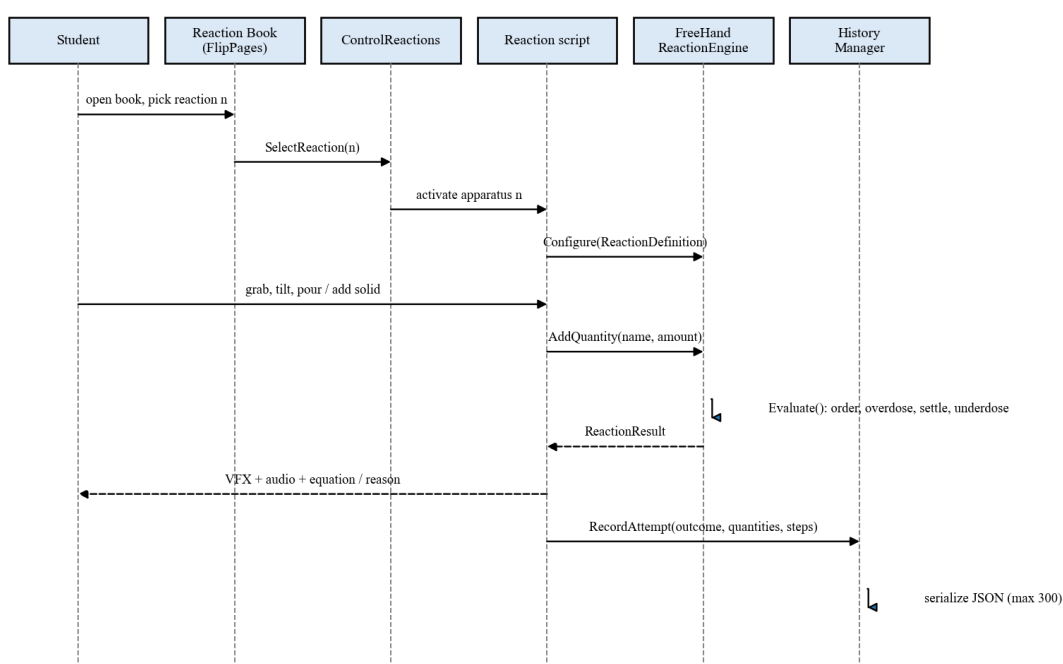


Figure 4.2: Sequence diagram - performing a free-hand experiment

Figure 4.3 shows the sequence for an AI question. The `InLabAssistantController` first asks the `ExperimentContextProvider` for the current reaction, quantities, targets, verdict and attempt counts, and sends the question together with this context to the Convai backend. If the request succeeds the reply text and audio are returned; if credentials are missing, the network fails or the quota is exhausted, the same question is answered by the `ChemistryKnowledgeBase` from the game's own data. In both cases the answer is spoken, shown in the assistant panel and recorded in the attempt history.

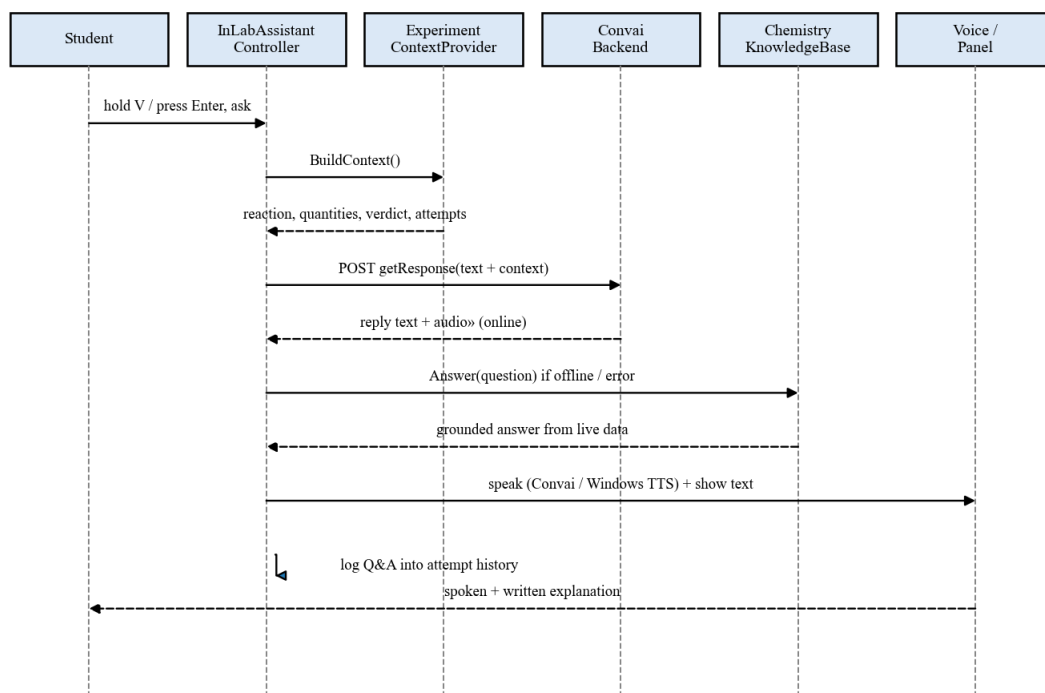


Figure 4.3: Sequence diagram - asking the context-aware AI assistant

4.2.3 State Diagram

The behaviour of the free-hand reaction engine is best described as a state machine (Figure 4.4). After Configure the engine waits for inputs. As soon as a pour starts it enters Pouring; overdose is checked on every frame, so pouring past the upper limit fails immediately even mid-pour. When all reagents are present and pouring has stopped the engine enters Settling and accumulates a settle timer; resuming a pour resets the timer and returns to Pouring. Only after 1.5 seconds of settling is underdose judged, which prevents a student who pauses between reagents from being failed unfairly. Adding a reagent out of order produces FailWrongOrder, and the heating reactions add procedure checks such as heating time and, for calcium carbonate, fitting the balloon before heating. Every terminal state can be reset with F5.

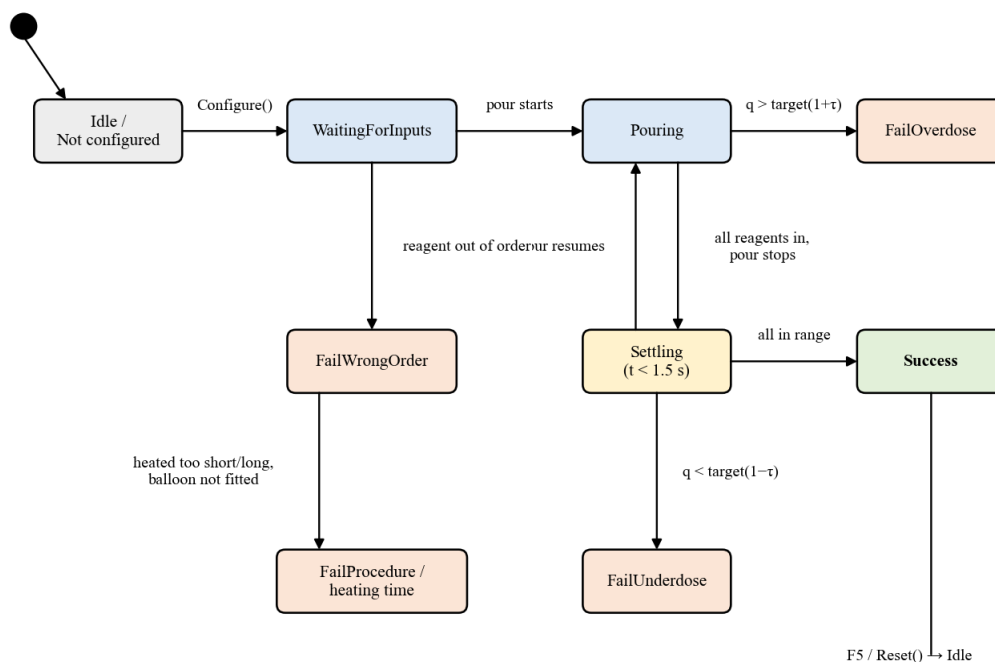


Figure 4.4: State diagram of the free-hand reaction engine

4.2.4 Data Flow Diagrams

Figure 4.5 is the context-level (level 0) DFD. Atomix exchanges data with three external entities: the student, who provides VR or keyboard-and-mouse input, voice and typed questions and receives the 3D scene, feedback, speech, graphs and scores; the Convai cloud service, which receives a question with its context and returns reply text and audio; and the local file system, which stores the experiment history and exported reports.

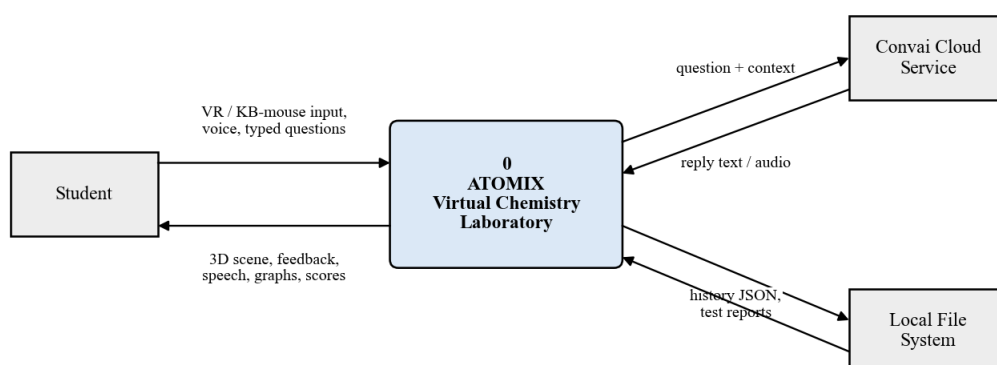


Figure 4.5: Level 0 DFD (context diagram)

Figure 4.6 decomposes the system into six processes. Input handling (1.0) unifies XR and desktop input. Reaction selection and learning (2.0) reads recipes and video clips from data store D1. Free-hand reaction evaluation (3.0) receives pour and add events, produces verdicts, and feeds live context to AI assistance (4.0). Assessment and coin economy (5.0) issues test tasks to the evaluation process and stores coins and settings in PlayerPrefs (D2). History, scoring and graphs (6.0) records attempts and AI exchanges in the history file (D3).

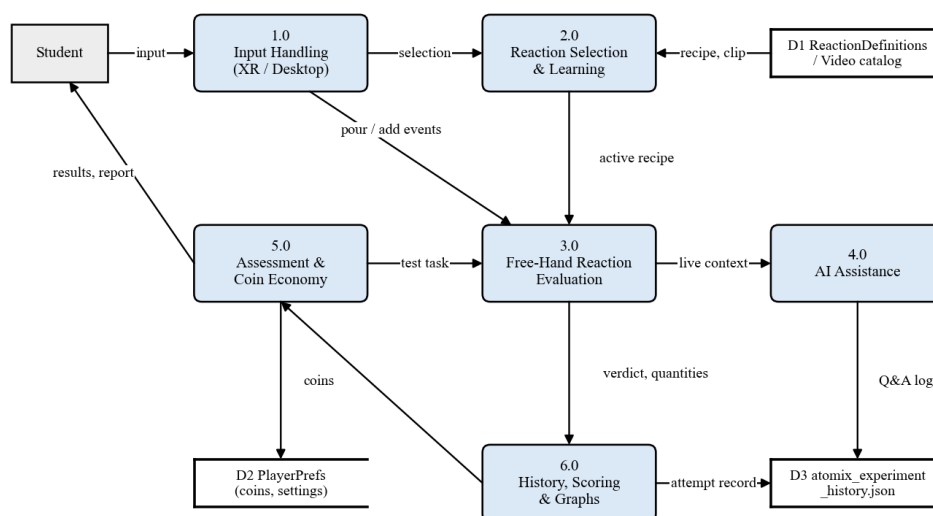


Figure 4.6: Level 1 DFD

Figure 4.7 refines process 3.0 to level 2. Reagents are registered with target, unit and tolerance (3.1); quantities are accumulated live (3.2); order and overdose are checked every frame (3.3); a settle timer runs once everything has been added (3.4); underdose and procedure rules are then checked (3.5). A successful result is scored for accuracy and stars (3.6), and every verdict is turned into visual, audio and text feedback by the feedback director (3.7) before being passed to history and to the AI context.

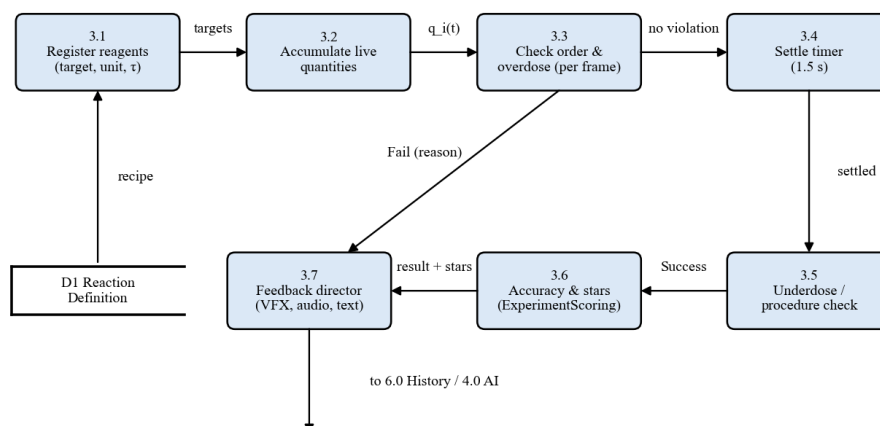


Figure 4.7: Level 2 DFD of the reaction-evaluation process

4.2.5 Class Diagram

Figure 4.8 shows the principal classes. ControlReactions owns the eight reaction objects and selects one at a time. A ReactionDefinition ScriptableObject holds the display name, tolerance, order rule and reagent list, and configures the FreeHandReactionEngine, a plain C# class whose only Unity dependency is the frame time. Each of the eight reaction scripts uses the engine, notifies the in-lab assistant, and is scored by the static ExperimentScoring class. Attempts are recorded by the ExperimentHistoryManager. The InLabAssistantController uses the Convai backend and falls back to the static ChemistryKnowledgeBase. AtomixCoinBank encapsulates prices, bonuses and rank thresholds for the testing scene.

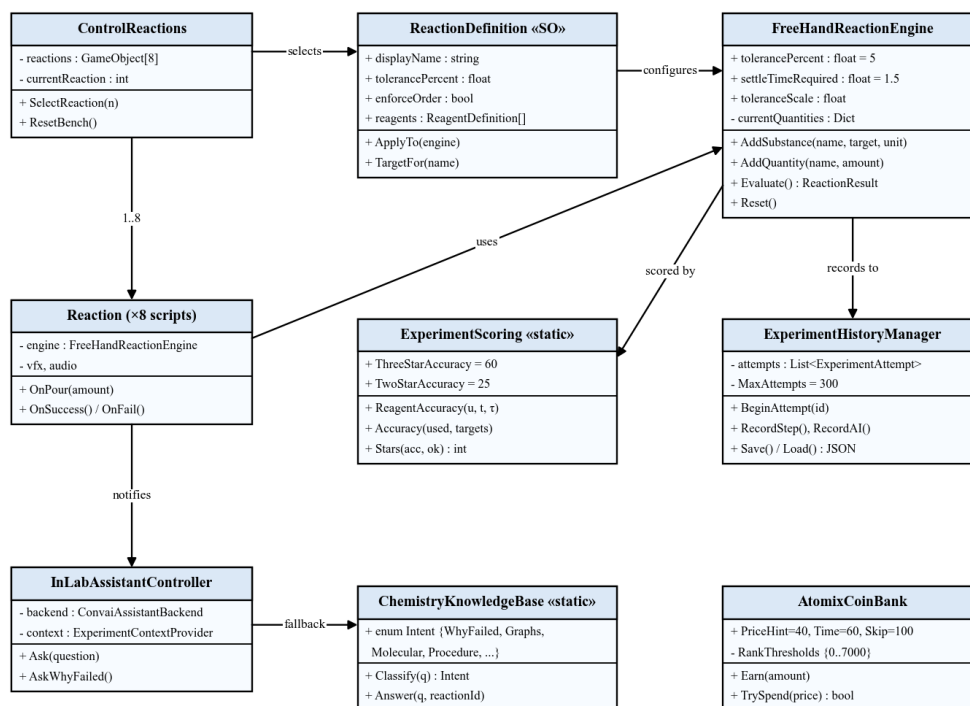


Figure 4.8: Class diagram of the core subsystems

4.2.6 Activity Diagram

Figure 4.9 shows the overall activity flow. From the main menu the user chooses one of three modes. In practice mode the Reaction Book leads to a learn-or-perform decision; performing ends either in success, which unlocks the scientific graphs and a molecular replay, or in failure, which shows the reason, invites a question to the AI and allows a retry. In test mode the user chooses the number of tasks, completes randomized practical and theory tasks under a timer, and receives coins, a report card and the exported report. In assistant mode the user converses with the AI character directly.

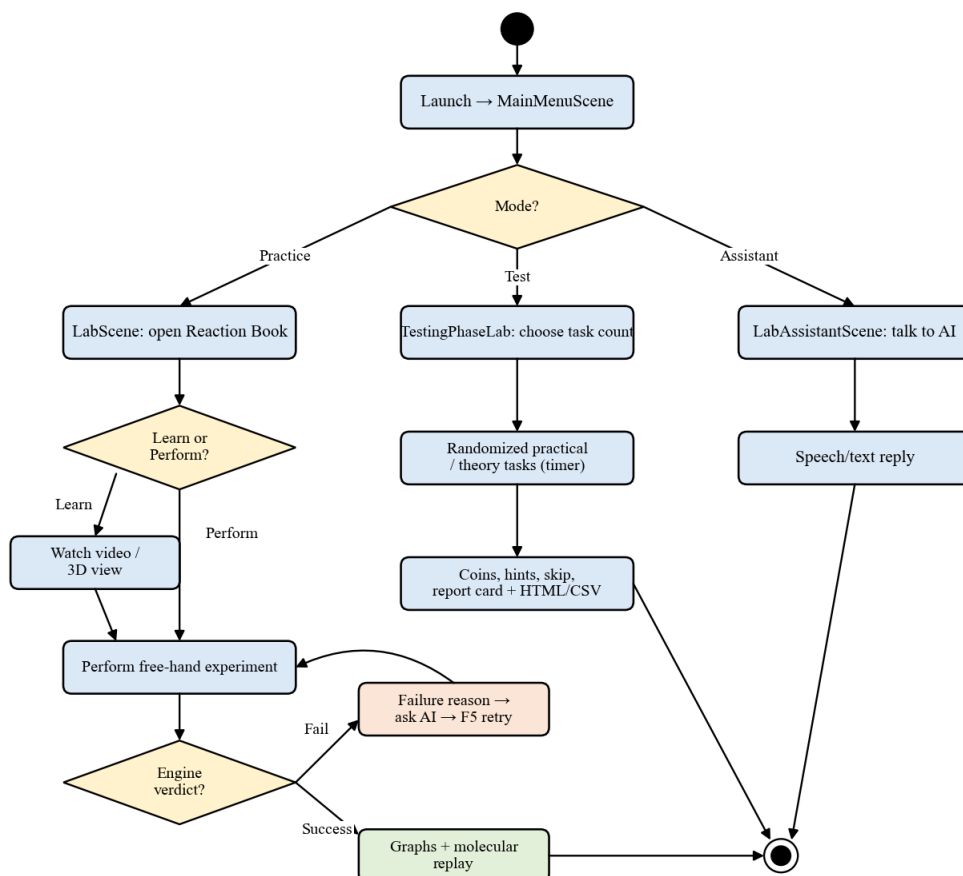


Figure 4.9: Activity diagram of the user journey

4.3 Scene Architecture

Atomix is organised into four Unity scenes. Separating the scenes keeps memory use low on a standalone headset and lets each scene own only the systems it needs. Table 4.2 lists the scenes and their responsibilities.

Table 4.2: Scene architecture

Scene	Responsibility	Key systems
MainMenuScene	Entry point; mode selection; settings	MainMenu, world-space canvas, async loader
LabScene	Practice laboratory	Reaction Book, ControlReactions, eight reactions, FreeHandReactionEngine, history, graphs, in-lab AI, periodic table
TestingPhaseLab	Assessment	Randomize, TheoreticalTasksManager, CountdownTimer, ExamSession, AtomixCoinBank, ExamReportExporter
LabAssistantScene	Dedicated AI tutor with avatar	LabAssistantCharacter, InLabAssistantController, avatar animator

4.4 Design Decisions

Several design decisions shaped the system. First, **reaction rules are data, not code**: targets, units, tolerances and order are stored in eight ReactionDefinition assets, so a teacher-facing change such as widening a tolerance needs no programming. Second, **one engine judges every reaction**, in both the practice laboratory and the testing scene, so behaviour is consistent and can be verified once. Third, **the desktop mode is injected at run time** by DesktopBootstrap using a RuntimeInitializeOnLoadMethod hook, so the VR scenes did not have to be duplicated. Fourth, **the AI has two brains**: a cloud model for open conversation and a grounded offline knowledge base that is, for this bench, more precise because it reads the live measurements. Fifth, **failure is a first-class outcome**: every failure path has a chemistry-specific message and a sound designed not to punish, because a learner is expected to fail many times while practising.

CHAPTER 5: METHODOLOGY

5.1 Introduction

This chapter explains how Atomix works internally. It first presents the development methodology and the layered system architecture, then details the algorithms that make the laboratory quantity-aware, the scoring model, the AI routing and intent classification, the molecular animation, the thermodynamic model behind the graphs and the assessment economy. Mathematical formulations are given where the behaviour is numerical.

5.2 Development Methodology

Atomix was developed with an incremental, phase-wise process (Figure 5.1). An initial pass through requirement analysis, laboratory design, 3D modelling, interaction design, reaction-engine development, AI integration, assessment and verification produced a working prototype. The prototype was then extended through ten feature-and-fix iterations (internally called Tasks 1 to 10), each of which began with an analysis of the existing code, implemented a focused set of changes, compiled them against the real Unity assemblies and executed automated checks against the shipped logic before being merged. Play-testing feedback from each iteration (for example, glassware that could not be grabbed in the testing scene, or the room tone sounding like a running tap) fed the next iteration.

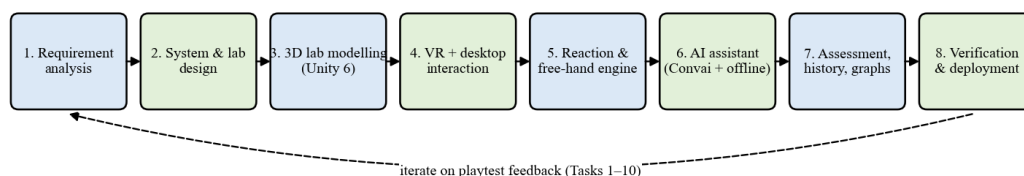


Figure 5.1: Phase-wise development methodology

5.3 System Block Diagram

Figure 5.2 shows the layered architecture of Atomix. It is presented here, before the algorithms, because each algorithm in this chapter lives in one of these layers.

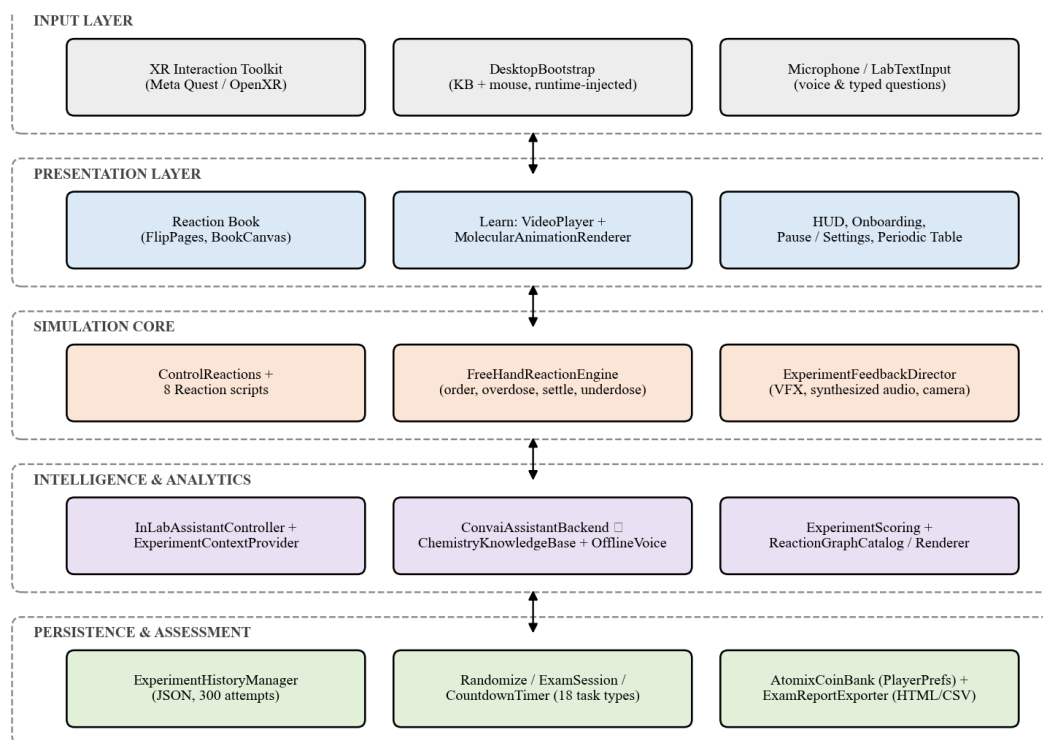


Figure 5.2: System block diagram of Atomix

Input layer. On a headset, the XR Interaction Toolkit provides grab, socket and locomotion interactions. Without a headset, DesktopBootstrap injects a first-person controller, a centre-screen crosshair, ray-based object interaction and keyboard bindings. Questions arrive from the microphone (push-to-talk) or from LabTextInput, which captures typed text and globally gates the keyboard so that typing a question does not move the player.

Presentation layer. The Reaction Book (FlipPages, BookCanvasManager) lists the reactions by category. The learning panel plays recorded MP4 clips through Unity's VideoPlayer or renders a live molecular animation to a RenderTexture. HUD elements include the objective line, onboarding card, measurement tooltip and tracker, the pause and settings menu and an interactive 118-element periodic table.

Simulation core. ControlReactions activates the chosen reaction; each of the eight reaction scripts translates object interactions into quantities for the FreeHandReactionEngine, and the ExperimentFeedbackDirector turns verdicts into effects, synthesized sounds and camera feedback.

Intelligence and analytics. The InLabAssistantController builds context through ExperimentContextProvider and routes questions to the Convai backend or the offline

ChemistryKnowledgeBase. ExperimentScoring rates attempts and the ReactionGraphCatalog and Renderer draw the scientific graphs.

Persistence and assessment. ExperimentHistoryManager stores attempts as JSON; Randomize, ExamSession and CountdownTimer run the test; AtomixCoinBank persists the wallet in PlayerPrefs; ExamReportExporter writes the HTML and CSV report.

5.4 Reaction Recipes

Every experiment is described by a recipe stored in a ReactionDefinition asset. Table 5.1 lists the recipes exactly as shipped. Liquids are measured in millilitres, solids in grams and heating in seconds, and every recipe enforces the order of addition. Tolerances are tighter for the violent alkali-metal reactions (5 %) and looser for the heating reactions (15 %), where timing by hand is inherently less precise.

Table 5.1: Reaction recipes used by the free-hand engine

#	Reaction	Reagents and targets (in order)	Tolerance τ
1	$2\text{Na} + 2\text{H}_2\text{O} \rightarrow 2\text{NaOH} + \text{H}_2$	Water 50 ml; Sodium 5 g	$\pm 5 \%$
2	$\text{H}_2\text{SO}_4 + \text{CuO} \rightarrow \text{CuSO}_4 + \text{H}_2\text{O}$	H_2SO_4 20 ml; CuO 8 g	$\pm 8 \%$
3	$\text{HCl} + \text{NaHCO}_3 \rightarrow \text{NaCl} + \text{H}_2\text{O} + \text{CO}_2$	HCl 15 ml; NaHCO_3 12 g	$\pm 10 \%$
4	$2\text{K} + 2\text{H}_2\text{O} \rightarrow 2\text{KOH} + \text{H}_2$	Water 50 ml; Potassium 3 g	$\pm 5 \%$
5	$2\text{Al} + 3\text{I}_2 \rightarrow 2\text{AlI}_3$	Aluminium 5 g; Iodine 15 g; Water drops 2 ml	$\pm 10 \%$
6	$\text{CaO} + \text{H}_2\text{O} \rightarrow \text{Ca(OH)}_2$	Water 30 ml; CaO 15 g	$\pm 8 \%$
7	$\text{CaCO}_3 \rightarrow \text{CaO} + \text{CO}_2$ (heat)	Heating 10 s (balloon fitted first)	$\pm 15 \%$
8	$2\text{FeSO}_4 \rightarrow \text{Fe}_2\text{O}_3 + \text{SO}_2 + \text{SO}_3$ (heat)	Heating 10 s	$\pm 15 \%$

5.5 Free-Hand Reaction Evaluation Algorithm

The FreeHandReactionEngine replaces the original fixed-sequence triggers. Let reagent i have target T_i , tolerance τ_i (per cent) and live quantity $q_i(t)$. A difficulty level multiplies every tolerance by a scale factor s , so the accepted range of each reagent is given by Equations (5.1) and (5.2).

$$q_i^{\min} = T_i (1 - s \cdot \tau_i / 100) \quad (5.1)$$

$$q_i^{\max} = T_i (1 + s \cdot \tau_i / 100) \quad (5.2)$$

where the scale factor s is 1.5 for the Guided level, 1.0 for Standard and 0.6 for Expert.

For a liquid, q_i is integrated from the pour rate, which depends on how far the vessel is tilted: $dq_i/dt = r(\theta)$ while θ exceeds the pour threshold. For a solid, q_i increases in discrete additions, and for a heating step q_i is the accumulated time spent over the lit burner. The engine evaluates the following algorithm on every frame:

1. If the engine is not configured or has already produced a verdict, return the stored state.
2. **Order check.** If order is enforced and a reagent has been added while an earlier reagent in the recipe is still absent, return **FailWrongOrder** with the recipe's order message.
3. **Overdose check.** For each reagent, if $q_i(t) > q_i^{\max}$, return **FailOverdose** immediately, with the reagent-specific overdose message (overdose is detected mid-pour).
4. If any vessel is still pouring, reset the settle timer, set the state to Pouring and return InProgress.
5. If no reagent, or not every reagent, has been added, reset the settle timer, set the state to WaitingForInputs and return InProgress.
6. **Settling.** Otherwise add the frame time Δt to the settle timer t_s ; while $t_s < 1.5$ s return InProgress (state Settling).
7. **Underdose check.** For each reagent, if $q_i < q_i^{\min}$, return **FailUnderdose** with the reagent-specific message.
8. **Procedure check.** For heating reactions, verify heating duration and procedure preconditions (for CaCO_3 , the balloon must be fitted before heating); otherwise return the procedure failure.
9. Return **Success**; compute accuracy and stars (Section 5.6); record the attempt.

The settling rule can be written formally as Equation (5.3), where $P(t)$ is true while any pour is in progress and $A(t)$ is true when every reagent has a non-zero quantity:

$$t_s \leftarrow (t_s + \Delta t) \cdot [A(t) \wedge \neg P(t)] \quad (5.3)$$

Here $[\cdot]$ is 1 when the condition holds and 0 otherwise, so the timer restarts whenever pouring resumes; underdose is judged only once $t_s \geq 1.5$ s.

Figure 5.3 visualises the accepted windows for every reagent at the three difficulty levels. The Guided level widens each window by half for a first attempt, while the Expert level, unlocked by passing Standard, narrows it to 60 % and hides the targets.

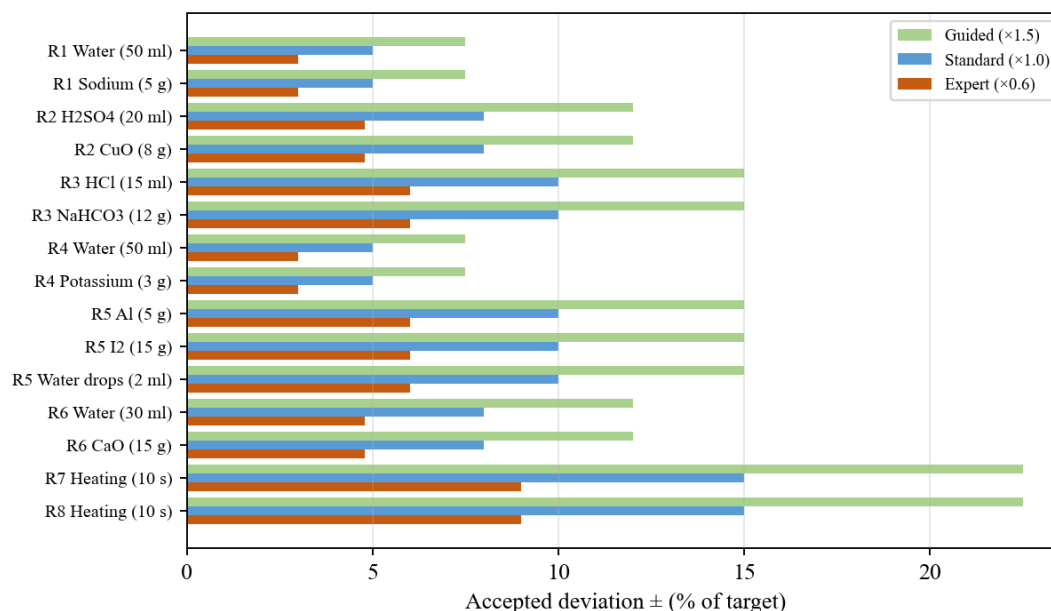


Figure 5.3: Accepted deviation per reagent at the three difficulty levels

Table 5.2 lists the verdicts the engine can return and the feedback associated with each.

Table 5.2: Engine verdicts and feedback

Verdict	Trigger	Feedback to the learner
Success	All quantities within range after settling; procedure correct	Reaction effect, sound, balanced equation, accuracy and stars, graphs unlocked
FailOverdose	Any $q_i > q_i^{\max}$ (checked every frame)	Reagent-specific chemistry reason, e.g. excess reagent pushes mixture away from the balanced equation
FailUnderdose	Any $q_i < q_i^{\min}$ after 1.5 s settling	Which reagent was short and its chemical consequence
FailWrongOrder	Reagent added before an earlier one	Why the order of addition matters for this reaction
Procedure failure	Heating too short/long; balloon not fitted (CaCO_3)	e.g. CO_2 escaped into the room instead of being collected

5.6 Accuracy and Star Rating Model

Before scoring existed, a titration that landed exactly on the mark and one that scraped in at the edge of the tolerance produced the same result. ExperimentScoring now converts every successful attempt into an accuracy percentage. Each reagent scores 1

when exactly on target, falling linearly to 0 at the edge of its accepted range (Equation 5.4); the experiment scores its least accurate reagent (Equation 5.5).

$$a_i = \text{clamp}(1 - |q_i - T_i| / (T_i \cdot \tau_i / 100), 0, 1) \quad (5.4)$$

$$A = 100 \cdot \min_i a_i \quad (5.5)$$

The minimum rather than the mean is used so that one careful measurement cannot compensate for a careless one, and so that a reagent added as a fixed lump (such as the sodium block, which is always exact) cannot inflate the score. Scoring always uses the recipe's Standard tolerance, so three stars mean the same precision on every difficulty level. Stars are then assigned by Equation (5.6).

$$\text{stars} = 3 \text{ if } A \geq 60; \ 2 \text{ if } 25 \leq A < 60; \ 1 \text{ otherwise} \quad (5.6)$$

A failed attempt always receives zero stars, and a successful attempt always receives at least one.

Figure 5.4 plots Equation (5.4) for reaction 2 (H_2SO_4 , target 20 ml, $\tau = 8\%$). The accepted window is 18.4-21.6 ml; three stars require every reagent within 40 % of the allowed deviation, i.e. within ± 0.64 ml of the target.

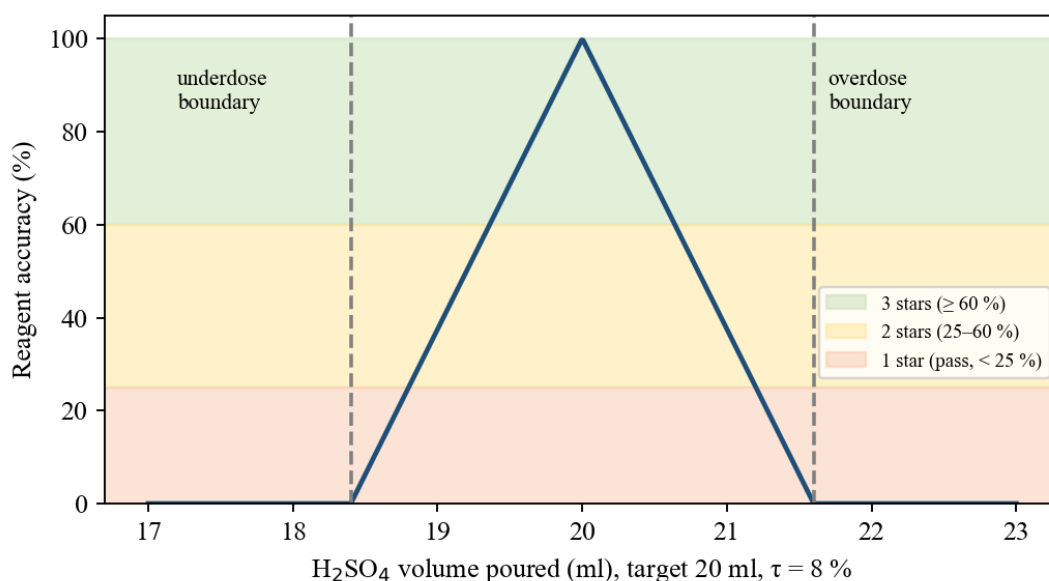


Figure 5.4: Reagent accuracy function and star bands for H_2SO_4 in reaction 2

5.7 AI Assistant: Context Building, Routing and Intent Classification

Figure 5.5 shows how a question is processed. Questions can come from push-to-talk (V), typed input (Enter), the "why did it fail" shortcut (Y), or the "Ask AI" buttons on the graph and molecular panels. The context builder serialises the current reaction, the

measured quantities, the targets and accepted ranges, the verdict and failure reason, and the number of previous and successful attempts. If credentials and network are available, the question and context are posted to the Convai character endpoint, which returns text and synthesized speech. On missing credentials, network error, time-out or exhausted quota, the question is routed to the offline ChemistryKnowledgeBase.

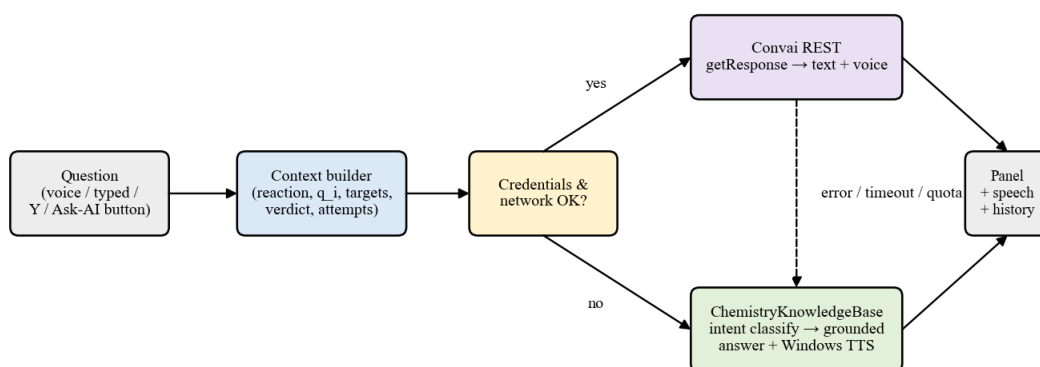


Figure 5.5: Dual-path routing of AI questions

The offline knowledge base is a grounded lookup rather than a language model. It classifies the question into an intent by keyword scoring and answers from the game's own data. If $S_k(q)$ is the number of keyword matches of question q for intent k , the chosen intent is given by Equation (5.7):

$$k^* = \operatorname{argmax}_k S_k(q); \quad k^* = \text{Overview if } \max_k S_k(q) = 0 \quad (5.7)$$

Table 5.3 lists the intents together with an example question and the data each answer is drawn from.

Table 5.3: Offline assistant intents and answer sources

Intent	Example question	Answer source
WhyFailed	Why did my experiment fail?	Engine verdict, offending reagent, used vs. target
Quantities	How much water do I need?	ReactionDefinition targets and ranges
Procedure	What do I do next?	Recipe order and procedure steps
WhatHappened	What happened in the beaker?	Reaction facts and products
Molecular	What is the yellow dot?	Molecular scene stage captions
Graphs	Explain the energy graph	ReactionGraphCatalog values (ΔH , E_a , ΔS , ΔG)
Calculation	Is this reaction spontaneous?	Derived Gibbs energy and crossover temperature

Intent	Example question	Answer source
Safety	Is this dangerous?	Per-reaction safety facts
Progress	How am I doing?	Experiment history counts
Overview	(no keyword matched)	Reaction summary

Because it reads live measurements, the offline path can give precise, personal answers such as "you poured 27.4 ml where the equation needs 20.0, a 37 % overdose", which a generic cloud model cannot do without the context. Offline replies are spoken through the Windows speech synthesiser; on other platforms they are shown as text.

5.8 Molecular Animation Model

The live 3D view is authored entirely in code: MolecularSceneCatalog defines, for each of the eight reactions, a sequence of stages containing atoms (element, position, CPK colour) and bonds, and MolecularAnimationRenderer interpolates between stages and renders the result into a RenderTexture shown on the learning panel. Three visual encodings make the chemistry explicit: a bond that thins and turns red is breaking, a bond that thickens in green is forming, and a travelling yellow dot is an electron being transferred. Electron transfer is shown only for the redox reactions (1, 4, 5 and 8); the non-redox reactions such as 2, 3 and 6 deliberately show none, and the caption of reaction 2 says so. Heating stages shake the molecule and warm the background, and a step counter and progress bar show the position in the sequence. The automated checks for this module verify atom conservation between stages and bond lengths.

5.9 Thermodynamic Model for Scientific Graphs

After a successful experiment the learner can open three graphs: an energy profile, an exothermic/endothermic diagram and an entropy comparison. Each reaction has one entry in the ReactionGraphCatalog holding its enthalpy change ΔH , activation energy E_a and entropy change ΔS . ΔH and ΔS were derived from standard thermodynamic data for the equations used; activation energies are literature-plausible values exposed as editable fields. The catalog derives further quantities, given by Equations (5.8) to (5.10).

$$\Delta G(T) = \Delta H - T \cdot \Delta S / 1000 \quad (5.8)$$

$$T^* = 1000 \cdot \Delta H / \Delta S \quad (5.9)$$

$$E_{a,rev} = E_a - \Delta H \quad (5.10)$$

In these equations ΔH , ΔG and E_a are in kJ/mol and ΔS in J/mol·K (hence the factor 1000); T^* is the crossover temperature at which ΔG changes sign, meaningful only when ΔH and ΔS have the same sign; and $E_{a,rev}$ is the activation energy of the reverse reaction.

Figure 5.6 shows energy profiles drawn from these values for an exothermic reaction (sodium and water) and an endothermic one (calcium carbonate decomposition). The contrast between a 10 kJ/mol barrier and a 185 kJ/mol barrier explains, at a glance, why the first reaction starts on contact while the second needs a burner.

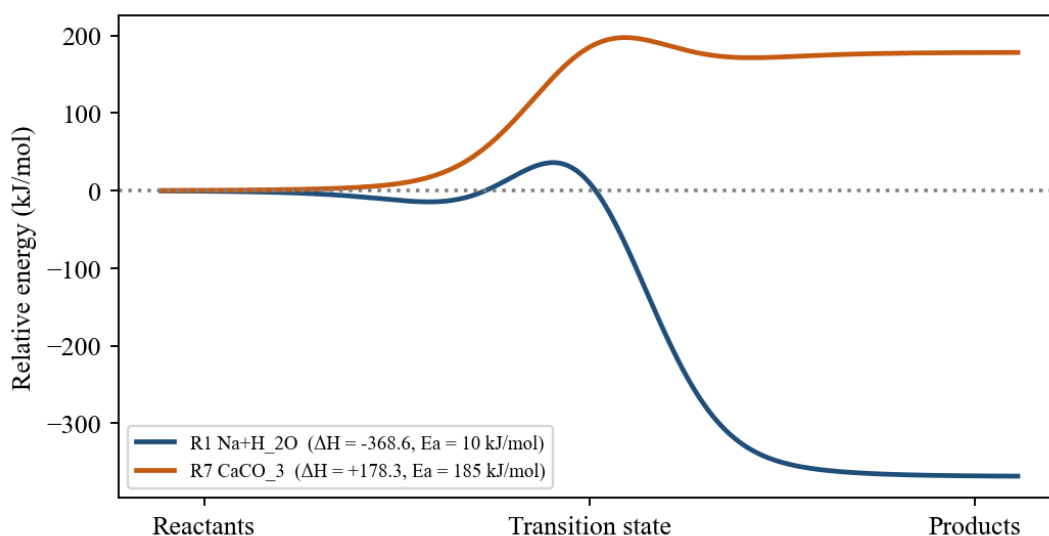


Figure 5.6: Energy profiles generated from catalog data for reactions 1 and 7

5.10 Assessment and Coin Economy

The testing scene draws tasks from eight practical tasks and ten theory questions (eighteen task types). A setup screen asks for the run length (3, 5, 8, 10 or all), clamped to what the selected mix contains: 10 for theory only, 8 for practical only and 18 for both. The countdown is held until a choice is made. Practical tasks are judged by the same free-hand engine as the laboratory, with targets hidden. Coins are earned per task, between 5 and 100 according to the time remaining, with bonuses for a first-ever clear, streaks and an unaided run, and can be spent on help. The lifetime total L and spent total S persist in PlayerPrefs; the balance and rank are computed by Equations (5.11) and (5.12), where $\theta = \{0, 500, 1500, 3500, 7000\}$ are the rank thresholds. Table 5.4 lists all parameters of the economy.

$$B = \max(0, L - S) \quad (5.11)$$

$$rank = \max\{r : L \geq \theta_r\}, \quad progress = (L - \theta_r) / (\theta_{r+1} - \theta_r) \quad (5.12)$$

Table 5.4: Coin-economy parameters

Parameter	Value
Earning per task	5-100 coins, scaled by time left
Bonuses	First-ever clear +25; 3-streak +30; 5-streak +60; perfect unaided run +50
Purchases	Hint 40 (F2); +30 seconds 60 (F3); skip task 100 (F4)
Ranks	Apprentice (0), Lab Technician (500), Chemist (1500), Senior Chemist (3500), Lab Master (7000)
Hint content	Procedure only, never the quantity - the amount is what the test measures

5.11 Frame-Rate-Independent Interaction Smoothing

Held objects and UI panels follow their targets smoothly. An early implementation used $x \leftarrow \text{Lerp}(x, \text{target}, k \cdot \Delta t)$, which is frame-rate dependent: a beaker followed the crosshair visibly harder at 144 Hz than at 60 Hz and overshoot below 1/k frames per second. It was replaced by the exponential form in Equation (5.13), which gives the same motion at any frame rate.

$$x \leftarrow x + (x_{\text{target}} - x) \cdot (1 - e^{-k \cdot \Delta t}) \quad (5.13)$$

5.12 Data Persistence

Each attempt is stored as an ExperimentAttempt record containing a unique identifier, reaction identifier and name, timestamp, duration, outcome, quantities used, target quantities, the ordered list of steps with their correctness, accuracy, stars and the AI questions and answers. The list is serialised to `atomix_experiment_history.json` in Unity's `persistentDataPath` and capped at 300 attempts, discarding the oldest. Coins, settings and achievements are stored in `PlayerPrefs`. Test reports are written to the Desktop, falling back to Documents and then to `persistentDataPath` if the Desktop is not writable.

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Introduction

Atomix was implemented in C# on Unity 6000.3.7f1 with the Built-in Render Pipeline. The project contains 110 C# scripts totalling 39,601 lines, four scenes, eight ReactionDefinition assets, seven recorded molecular MP4 clips and a set of free 3D laboratory models. This chapter describes the implementation of each module and presents the user interface. Screenshot placeholders marked [INSERT SCREENSHOT] must be replaced with captures from the final build before submission.

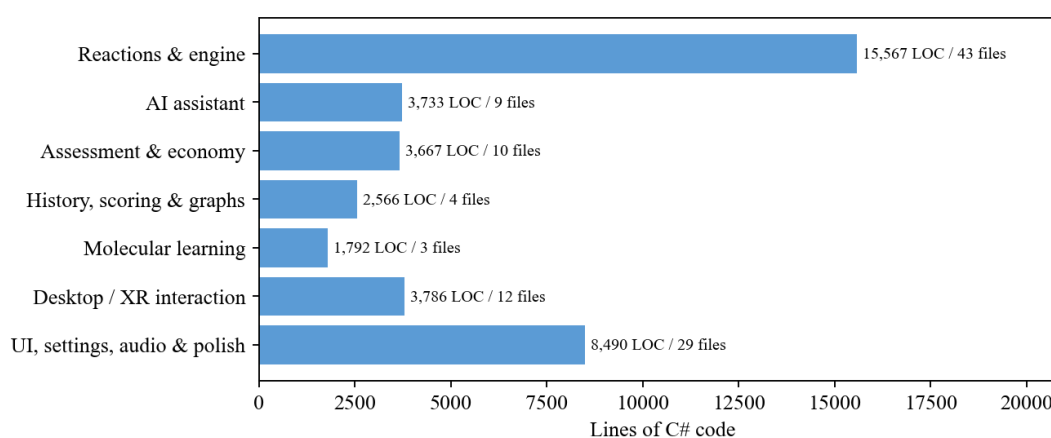


Figure 6.1: Distribution of the 39,601 lines of C# code across subsystems

Figure 6.1 shows how the code is distributed. The reaction scripts and engine form the largest group because each of the eight reactions has a laboratory script, a test-scene script and helpers for pouring and filling; the UI, settings, audio and presentation group is second, reflecting the effort spent on usability and comfort.

6.2 System Implementation

6.2.1 Laboratory Environment and Main Menu

The laboratory scene (Figure 6.2) was assembled from free 3D models of benches, cupboards, a blackboard, a sink, a microscope, a periodic-table poster and glassware such as Erlenmeyer flasks, Berzelius beakers, test tubes and pipettes. Lighting, ambient occlusion and a box-projected reflection probe give the glassware believable reflections of the room. The capture in Figure 6.2 shows the iron(II) sulfate experiment set up on the bench, with the assistant character's speech bubble explaining that the Bunsen

burner is lit by pressing its button. The main menu (Figure 6.3) is a world-space canvas offering the practice laboratory, the testing scene, the assistant scene, settings and exit.



Figure 6.2: The Atomix virtual laboratory with the FeSO₄ experiment set up and assistant guidance



Figure 6.3: Main menu

6.2.2 Reaction Book and Learn-Before-Perform

The Reaction Book (Figure 6.4) is a world-space book whose pages are turned with the controller, the mouse or the arrow keys; keys 1-8 select a reaction directly. Each reaction page offers LEARN and PERFORM and a difficulty selector (Guided, Standard, Expert). Choosing LEARN opens the learning panel. The ReactionLearningController looks up the reaction in the ReactionLearningVideoCatalog and plays the clip (Figure 6.5); the 3D VIEW button switches to the live molecular animation (Figure 6.6). The "Ask AI" button on this panel asks the assistant about the exact stage on screen.

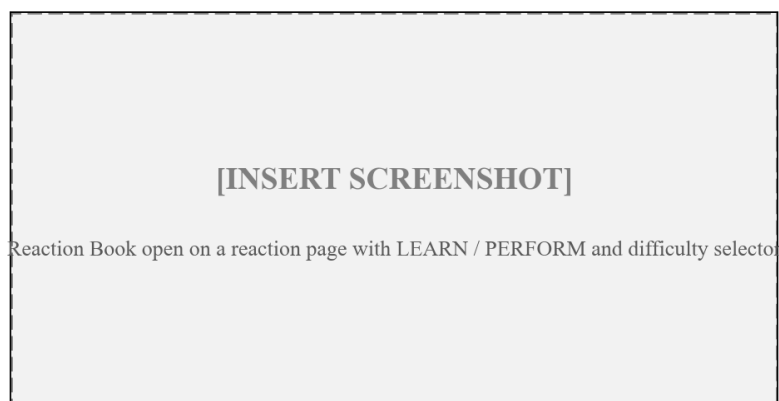


Figure 6.4: Reaction Book with LEARN / PERFORM and difficulty selection

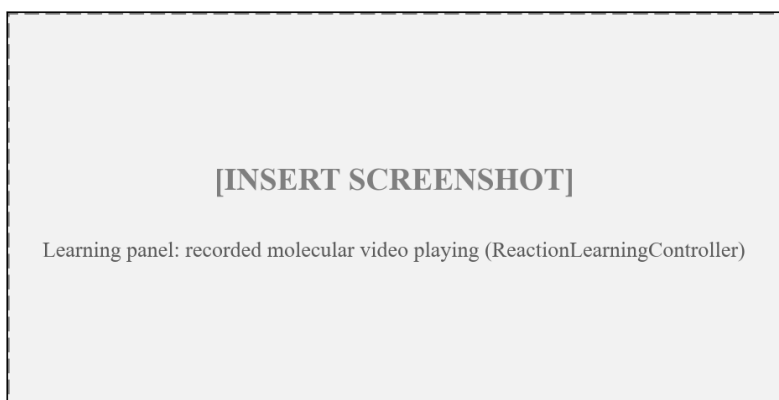


Figure 6.5: Learning panel playing a recorded molecular video

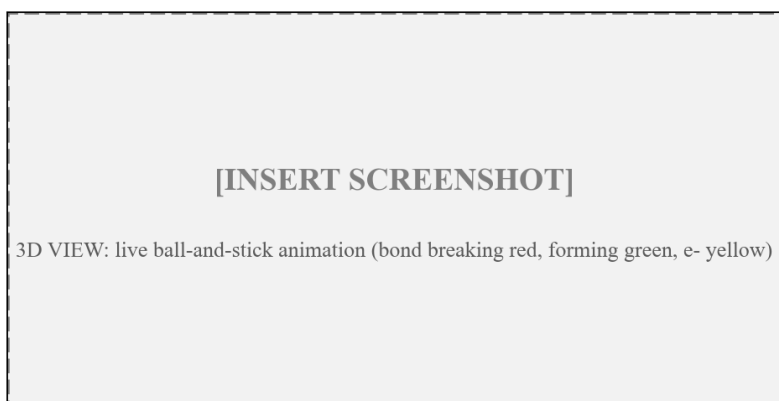


Figure 6.6: Live 3D molecular animation (bond breaking, forming and electron transfer)

6.2.3 Free-Hand Experimentation and Feedback

When PERFORM is chosen, ControlReactions enables only the apparatus of that reaction and configures the engine from its recipe. Pouring is implemented by monitoring the tilt of a held vessel; beyond a threshold angle a pour particle stream is emitted and the receiving vessel's quantity increases at a tilt-dependent rate. A floating measurement tooltip and a tracker panel (Figure 6.7) show the live amounts and, at

Guided and Standard levels, the targets and ranges. The L key toggles the label between full, compact and off. When the engine reaches a verdict, the ExperimentFeedbackDirector plays the success arpeggio or the gentle failure tones, shakes the camera on violent reactions, and shows either the balanced equation with accuracy and stars or a failure card with the chemistry-specific reason (Figure 6.8). Pressing F5 reloads the bench and re-selects the same experiment.

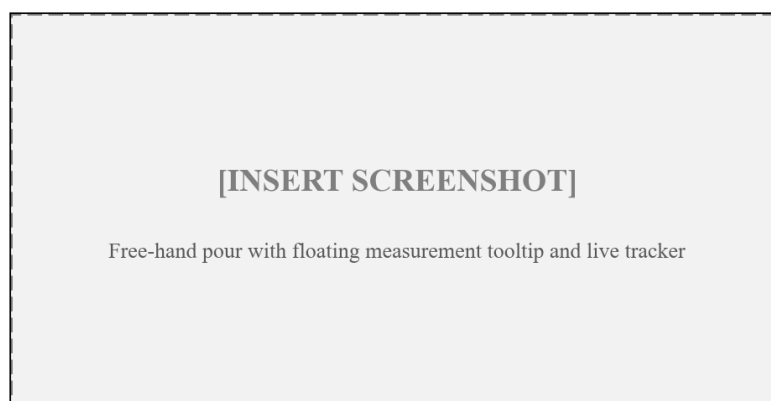


Figure 6.7: Free-hand pouring with live measurement tooltip and tracker

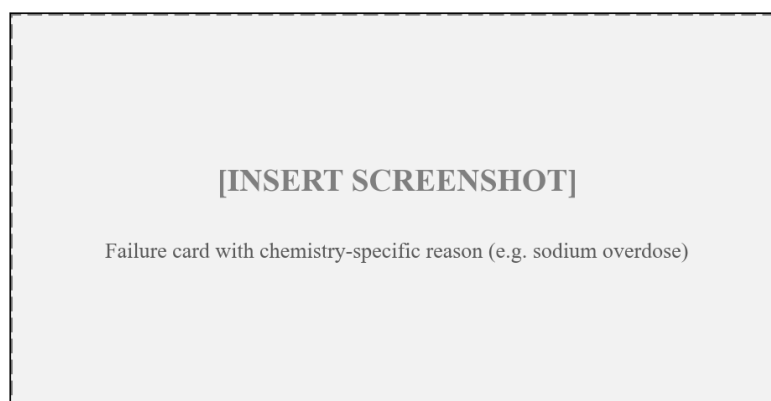


Figure 6.8: Chemistry-specific failure card

6.2.4 Experiment History and Scientific Graphs

The Tab key opens the Experiment History panel (Figure 6.9), which lists attempts newest first with colour coding for success, failure and abandoned attempts, a reaction filter, and expandable details showing used versus target quantities, every step and the AI exchanges. The F key opens the scientific graphs (Figure 6.10), drawn by ReactionGraphRenderer from the catalog: the energy profile with E_a , ΔH and the transition state; the enthalpy diagram with the direction of heat flow; and the entropy chart comparing all eight reactions on a common axis. Each graph has an "Ask AI to Explain" action that sends the graph's values to the assistant.

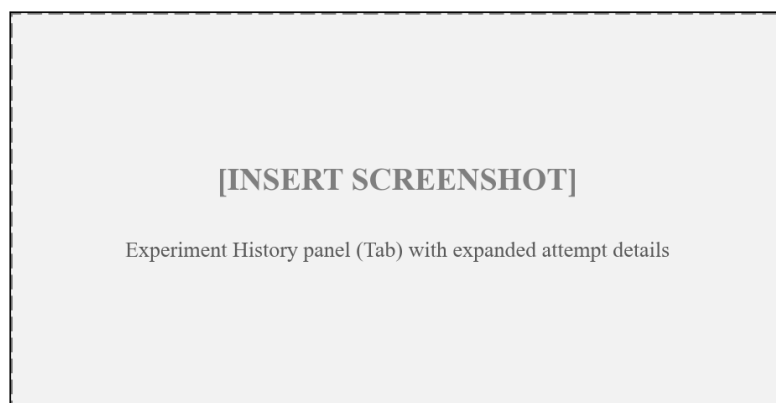


Figure 6.9: Experiment History panel

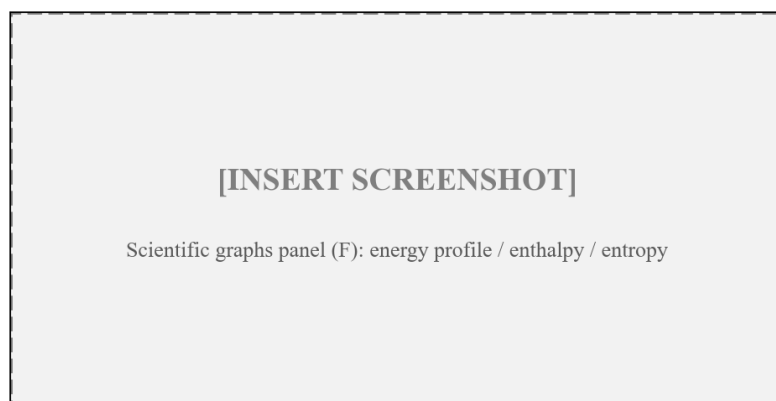


Figure 6.10: Scientific graphs panel

6.2.5 In-Laboratory AI Assistant

The assistant panel sits at the right edge of the view so that it does not obstruct the bench (Figure 6.11). Holding V records from the microphone; pressing Enter opens a text field managed by LabTextInput, which takes over the keyboard while open and is refused mid-pour so that a tipped beaker is never frozen. Y asks why the last experiment failed and M minimises the panel. ConvaiAssistantBackend posts the text and context to the Convai character endpoint using UnityWebRequest; the reply text is shown and the returned audio is played. The OfflineVoice class drives the Windows speech synthesiser for offline answers.



Figure 6.11: In-lab AI assistant answering a typed question

6.2.6 Testing Scene, Coin Economy and Reports

On entering TestingPhaseLab the ExamSetupUI asks for the number of tasks (Figure 6.12). Randomize selects practical reactions and TheoreticalTasksManager presents multiple-choice questions such as the colour of phenolphthalein in a basic solution or the gas produced by sodium bicarbonate and hydrochloric acid. ExamCoinHud shows the wallet and the F2/F3/F4 shop. ExamSession records every task, including skipped and timed-out ones, and at the end TestResultsUI shows a report card with grade, passed/total, coins and time. "Download this test report" writes a self-contained HTML file and a CSV file (Figure 6.13); "Retake the test" reloads the scene.



Figure 6.12: Testing scene setup and coin HUD

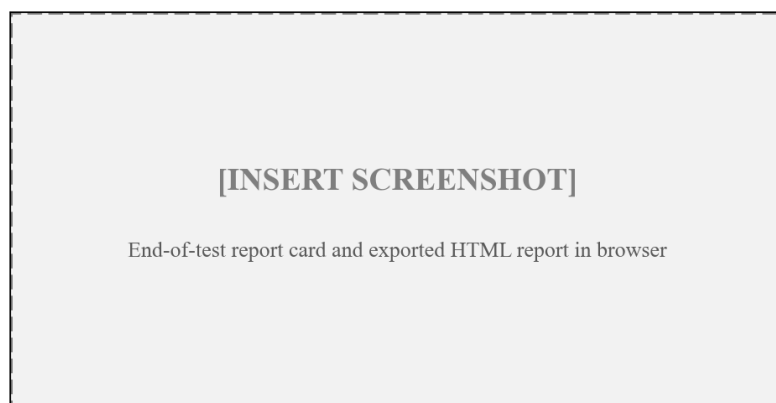


Figure 6.13: Report card and exported HTML test report

6.2.7 Periodic Table, Settings and Presentation

The P key opens an interactive periodic table of all 118 elements with an element card (Figure 6.14). F1 opens the pause menu (Figure 6.15) with settings, 13 achievements derived from the history, and a controls reference. Settings cover master volume and three audio buses, camera shake, walking motion (off by default to avoid motion sickness), post-processing, quality, full-screen mode and a frame-rate cap. All interface and handling sounds are synthesised at run time from a single D-major pitch palette, so no audio files had to be imported; grab sounds are pitched by object size. Scene changes load asynchronously behind a fade with a chemistry tip, replacing a blocking load that froze the window.

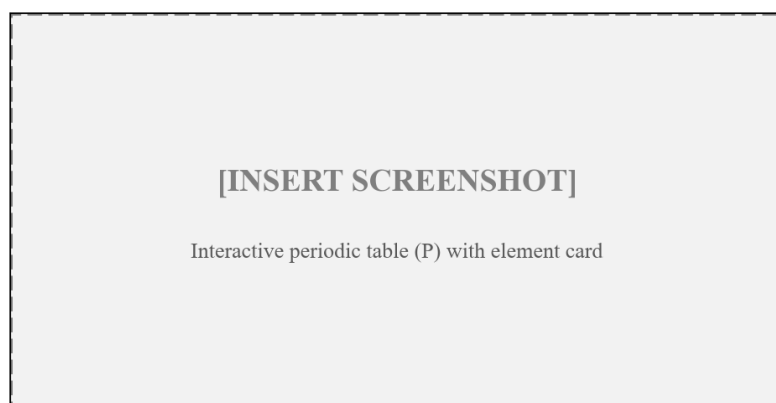


Figure 6.14: Interactive periodic table

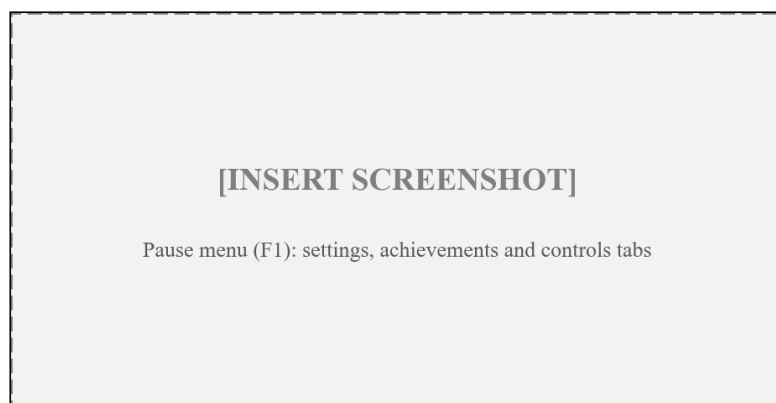


Figure 6.15: Pause menu with settings, achievements and controls

6.3 Desktop Mode Implementation

DesktopBootstrap uses Unity's RuntimeInitializeOnLoadMethod attribute so that, when no XR device is active, it runs after each scene loads and adds the desktop components: a FirstPersonController with walking, sprint and fly, a crosshair, an ObjectInteraction ray that grabs DesktopInteractable and ObjectGrabbable objects, a LabBoundary clamp that keeps the player inside the room, and a DesktopObjectSettler that lowers glassware authored slightly above the bench onto the surface below it. Table 6.1 lists the desktop controls.

Table 6.1: Desktop controls

Input	Action	Input	Action
W A S D / Mouse	Move / look	Tab	Experiment history
Left Shift	Sprint	F	Scientific graphs
Space / Ctrl	Fly up / down	P	Periodic table
Left click	Grab / release / interact	L	Measurement label mode
Q / E, Z / X, C	Rotate, tilt, roll held object	F1	Pause menu
T / R	Reset pose / release	V (hold)	Push-to-talk
B, Arrow keys	Book, flip pages	Enter	Type a question
1 - 8	Quick-select reaction	Y / M	Why failed / minimise AI
F5	Reset bench and retry	F2 / F3 / F4	Hint / +30 s / skip (test)

6.4 Key Scripts

Table 6.2 lists the most important of the 110 scripts and their responsibilities.

Table 6.2: Key scripts and responsibilities

Script	Responsibility
ControlReactions.cs	Reaction selection and laboratory set-up
FreeHandReactionEngine.cs (1,264 lines)	Shared quantity, order, overdose, underdose and settle logic
ReactionDefinition.cs	Data-driven recipe asset; configures the engine
Reaction*.cs, KOHReaction.cs, CaCO3Reaction.cs, Feso4Reaction.cs	Per-reaction interaction, effects and engine feeding
ExperimentScoring.cs	Accuracy percentage, stars and difficulty scales
ExperimentHistoryManager.cs / ExperimentHistoryUI.cs	Attempt persistence and browsing
InLabAssistantController.cs / ExperimentContextProvider.cs	In-lab AI and context building
ConvaiAssistantBackend.cs	REST calls to the Convai character endpoint
ChemistryKnowledgeBase.cs / OfflineVoice.cs	Offline intent-based answers and Windows speech
MolecularSceneCatalog.cs / MolecularAnimationRenderer.cs	Code-authored molecular stages and live rendering
ReactionGraphCatalog.cs / ReactionGraphRenderer.cs / ReactionGraphUI.cs	Thermodynamic data and graphs
Randomize.cs / ExamSession.cs / CountdownTimer.cs	Randomized timed assessment
AtomixCoinBank.cs / ExamCoinHud.cs / ExamReportExporter.cs	Wallet, shop, HTML/CSV report
DesktopBootstrap.cs / FirstPersonController.cs / LabTextInput.cs	Desktop mode and typed-input gate
AtomixAudio.cs / CameraJuice.cs / AtomixPostFx.cs	Synthesised audio, camera feedback, post-processing

6.5 VR Implementation

For the headset the scenes use an XR Origin with the XR Interaction Toolkit. Glassware carries grab interactables so that it can be picked up with either controller (or tracked hands with the Oculus hand models), world-space canvases receive ray interaction, and continuous or teleport locomotion moves the user. Because quantities are derived from vessel tilt and time, the same reaction scripts serve both the controller grab and the desktop grab without modification. The Quest build targets Android through the Oculus/OpenXR loader.

CHAPTER 7: RESULT ANALYSIS

7.1 Introduction

This chapter presents the results that can be stated with evidence. Atomix is a functional academic prototype; it has been verified by compilation and by automated checks that execute the shipped code, and its scientific content is quantified below. A controlled study of learning outcomes, usability scores or on-device frame rates has **not** yet been carried out, so no such figures are reported; placeholders are provided where the team is expected to add measured data.

7.2 Performance Measures

7.2.1 Build and Verification Results

Every development iteration was compiled against the real Unity 6000.3.7f1 assemblies, and the logic that does not depend on the engine (the reaction engine, scoring, molecular geometry, knowledge base, coin bank and audio synthesis) was extracted verbatim and executed outside Unity with a minimal time shim. Table 7.1 and Figure 7.1 summarise the results.

Table 7.1: Build and verification summary

Verification item	Result
Compilation against Unity 6000.3.7f1 assemblies	0 errors; 68 pre-existing warnings (CS0649/CS0414 in original scripts); 0 new warnings
Reaction-engine outcome matrix (8 test-scene experiments)	111 / 111 checks passed
Regression suite after molecular animation and offline AI	167 / 167 checks passed
Regression suite after coin economy and exam report	226 / 226 checks passed
Synthesised-audio suite	22 / 22 checks passed
Full Play-mode regression on headset and desktop	[INSERT: results of final Play-mode pass]

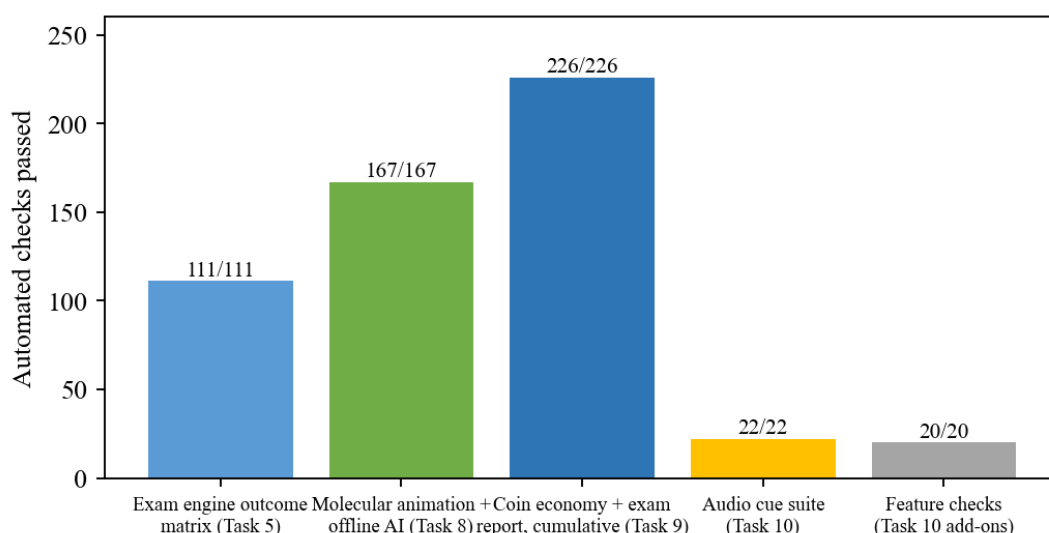


Figure 7.1: Automated verification checks passed per suite

The outcome matrix in Table 7.1 drove each test-scene experiment through correct amounts, overdose, underdose, reversed order and a pause mid-procedure, and confirmed that pausing never fails the learner and that the test scene never reveals target values. The molecular suite checked atom conservation and bond lengths across stages. All suites passed in full.

7.2.2 Content and Coverage Metrics

Table 7.2 and Figure 7.2 quantify the learning content delivered by the prototype.

Table 7.2: Content and coverage metrics

Metric	Value
Practical experiments (all quantity-aware)	8
Theory questions	10
Assessment task types	18
Reactions with live 3D molecular animation	8 of 8
Recorded molecular MP4 clips in project	7
Reaction categories in the book	6
Unity scenes	4
Achievements	13
Periodic-table elements	118
C# scripts / lines of code	110 / 39,601
Maximum stored attempts	300

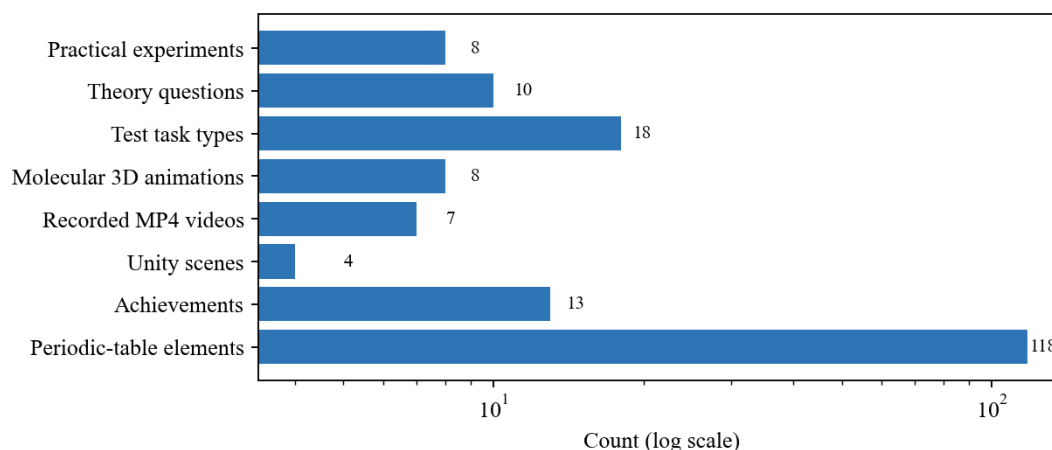


Figure 7.2: Content inventory of the Atomix prototype

7.2.3 Scientific Data Used by the System

Table 7.3 lists the thermodynamic values used by the graphs, together with the Gibbs free energy at 298.15 K computed with Equation (5.8). Figures 7.3 and 7.4 plot these values.

Table 7.3: Thermodynamic data per reaction and derived Gibbs energy

#	Type	ΔH (kJ/mol)	E_a (kJ/mol)	ΔS (J/mol·K)	ΔG_{298} (kJ/mol)	Spontaneous at 298 K
1	Exothermic	−368.6	10	−15.5	−364.0	Yes
2	Exothermic	−63.7	48	−43.0	−50.9	Yes
3	Endothermic	+31.4	45	+241.0	−40.5	Yes (entropy-driven)
4	Exothermic	−393.2	6	+44.7	−406.5	Yes
5	Exothermic	−627.6	85	−86.9	−601.7	Yes
6	Exothermic	−65.2	46	−26.3	−57.4	Yes
7	Endothermic	+178.3	185	+160.7	+130.4	No (needs heating)
8	Endothermic	+340.1	365	+377.4	+227.6	No (needs heating)

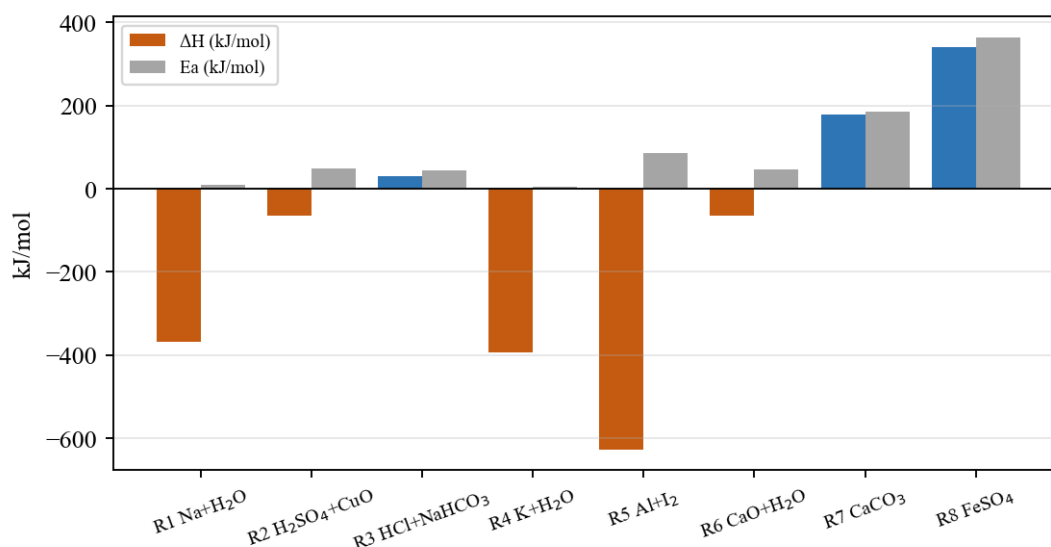


Figure 7.3: Enthalpy change and activation energy of the eight reactions

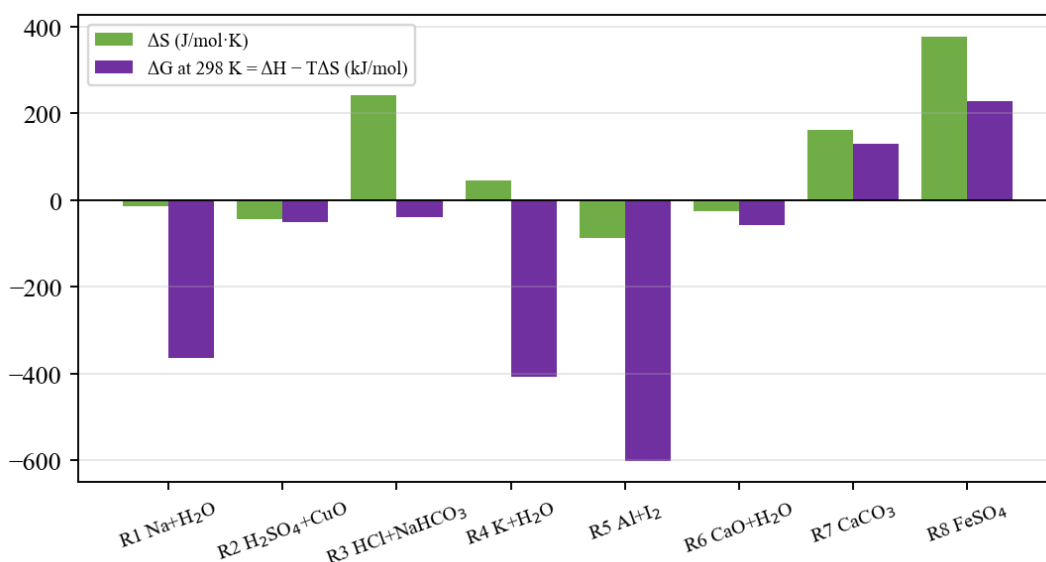


Figure 7.4: Entropy change and Gibbs free energy at 298 K

The data in Table 7.3 is consistent with what the learner observes in the laboratory. The two thermal decompositions (7 and 8) are the only reactions with positive ΔG at room temperature, which is exactly why they are the two experiments that require the Bunsen burner; Equation (5.9) gives crossover temperatures of about 1,110 K and 901 K respectively. Reaction 3 is endothermic yet spontaneous because of its large entropy gain from releasing CO_2 gas, which gives the teacher a concrete example of entropy-driven spontaneity. The alkali-metal reactions (1 and 4) combine very low activation energies with large negative ΔH , matching their violent behaviour on contact with water.

7.2.4 Requirement Traceability

Table 7.4 traces each functional requirement of Table 3.4 to its implementation and verification evidence.

Table 7.4: Requirement traceability

ID	Implemented by	Status / evidence
FR1	LabScene, 3D models, grab interactables	Implemented; Figure 6.2
FR2	FlipPages, BookCanvasManager, keys 1-8	Implemented
FR3	ReactionLearningController, MolecularAnimationRenderer	Implemented; 8/8 animations, 167/167 suite
FR4, FR5	FreeHandReactionEngine, ReactionDefinition	Implemented; 111/111 outcome matrix
FR6	ExperimentFeedbackDirector, LabRetryController (F5)	Implemented
FR7, FR8	InLabAssistantController, ConvaiAssistantBackend, ChemistryKnowledgeBase	Implemented; offline path verified in 167/167 suite
FR9, FR10	ExperimentHistoryManager, ExperimentScoring	Implemented
FR11	ReactionGraphCatalog / Renderer / UI	Implemented
FR12- FR14	Randomize, ExamSession, AtomixCoinBank, ExamReportExporter	Implemented; 226/226 suite
FR15	XR rig + DesktopBootstrap	Implemented; headset Play-mode pass [INSERT]

7.2.5 Measurements to be Added

The following measures are recommended for the final submission and the planned user study; they are left as placeholders because they have not yet been measured: [INSERT: average frame rate on Meta Quest and on the desktop test machine], [INSERT: scene load times], [INSERT: Convai average response latency versus offline response latency], and [INSERT: System Usability Scale scores and pre-test / post-test knowledge scores from a student study].

7.3 Screenshots

The principal screens of the system were presented in Section 6.2 (Figures 6.2 to 6.15). The laboratory capture in Figure 6.2 is taken from the running build; the remaining figures are placeholders to be replaced with captures from the final build.

7.4 Comparative Analysis

Table 7.5 compares Atomix with the representative systems from the literature survey on the dimensions identified in Section 2.2. The comparison is functional, based on the capabilities described in each work, because the surveyed systems were evaluated on different populations and metrics and their numerical results are not directly comparable.

Table 7.5: Comparison with existing systems

Feature	Virtual labs [1],[12]	VR systems [4],[13]	Molecular VR [5]	AI chatbots [7],[14]	Atomix
Platform	Desktop / web	VR	Projection CAVE	Web / messaging	Meta Quest + Windows desktop
Experiment judging	Step sequence	Scripted	None	None	Order, overdose, underdose, heating, with tolerance
Difficulty levels	Rare	No	No	No	Guided / Standard / Expert
Molecular view	Static images	Partial	Yes	No	Video + live animation for 8/8 reactions
AI help	No	No	No	Yes, generic	Context-aware, voice + text, offline fallback
Progress record	Scores	No	No	Chat log	300-attempt history with steps, accuracy, AI log
Thermodynamics	No	No	No	No	ΔH , E_a , ΔS , ΔG graphs with AI explanation
Assessment	Quizzes	No	No	No	18 task types, coins, HTML/CSV report

The comparison shows that Atomix is, to the best of our survey, the only system among those reviewed that combines quantity-aware experiment judging, molecular visualization, a context-aware AI assistant that works offline, attempt history, thermodynamic analysis and assessment in one application available both in VR and on a desktop. Its main weakness relative to the literature is evaluation: unlike [6], Atomix

has not yet been tested in a controlled study, so its effect on learning remains to be measured.

7.5 Limitations

The current prototype has the following known limitations: the application still identifies itself with the legacy product and company name of the original template, which is kept deliberately because renaming would reset players' saved data unless a migration is written; cloud AI requires valid credentials and internet, and offline answers cover only the eight reactions; offline speech is Windows-only; some visual substance state may remain after re-selecting an experiment; coins are stored per machine with no accounts or leaderboard; and the AI service key was committed to the repository history during development and must be rotated before the repository is shared.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Conclusion

This project designed and implemented Atomix, an AI-assisted virtual chemistry laboratory built in Unity 6 that runs on Meta Quest / OpenXR headsets and, through a run-time-injected keyboard-and-mouse mode, on ordinary Windows computers. Atomix integrates, in one application, eight quantity-aware inorganic experiments, a learn-before-perform workflow with recorded videos and live 3D molecular animations for all eight reactions, an in-laboratory AI assistant that knows the learner's live measurements and keeps working offline, a persistent experiment history with accuracy scores and star ratings, scientific graphs that connect each experiment to its enthalpy, activation energy, entropy and Gibbs free energy, and a randomized assessment scene with a persistent coin economy and exportable reports.

The central technical contribution is the data-driven free-hand reaction engine, which judges experiments by order of addition, overdose (instantly), underdose (after a settling period) and procedure, using per-reaction tolerances scaled by difficulty level, and which is shared by the practice laboratory and the test scene. The second contribution is the dual-path assistant whose offline brain gives grounded, personal explanations from the engine's own measurements. The implementation comprises 110 scripts and 39,601 lines of C#, compiles without errors, and passed every automated check that was run against the shipped logic (111/111, 167/167, 226/226 and 22/22). All functional requirements listed in Chapter 3 were implemented. As the literature warns, however, functional completeness is not the same as proven learning benefit; Atomix demonstrates technical and functional viability, and its educational effect must now be measured.

8.2 Future Scope

- **Learning-outcome study.** Conduct a controlled study with pre-tests and post-tests and usability questionnaires, comparing Atomix with conventional preparation, as recommended in [6] and [8].

- **Larger reaction library.** Add organic, titration and electrochemistry experiments; because recipes are data, many additions require only new ReactionDefinition assets and effects.
- **Teacher dashboard and accounts.** Synchronise history and test reports to a server so instructors can monitor classes and assign experiments; replace per-machine coins with accounts and leaderboards.
- **Richer offline AI.** Embed a small on-device language model grounded on the knowledge base, and extend offline speech beyond Windows.
- **Multiplayer laboratory.** Allow partners or a teacher to share the same bench in VR.
- **Performance and platform work.** Profile and optimise for standalone headsets, add mixed-reality passthrough mode, and support Android tablets and WebGL.
- **Accessibility and localisation.** Add Hindi and Marathi language support, subtitles for all narration and colour-blind-safe highlight schemes.

REFERENCES

- [1] S. Ramos, E. P. Pimentel, M. das G. B. Marietto, and W. T. Botelho, "Hands-on and virtual laboratories to undergraduate chemistry education: Toward a pedagogical integration," in Proc. 2016 IEEE Frontiers in Education Conf. (FIE), 2016, pp. 1-8, doi: 10.1109/FIE.2016.7757580.
- [2] M. Agarwal and S. Saha, "Learning chemistry through puzzle based game: Atoms to molecule," in Proc. 2011 9th IEEE Int. Conf. Emerging eLearning Technologies and Applications (ICETA), 2011, pp. 189-193, doi: 10.1109/ICETA.2011.6112613.
- [3] A. L. Sanchez-Lopez, O. R. Camacho-Ornelas, L. M. Lozano-Sanchez, M. F. Rivas-Gastelum, A. Parra-Cordova, and Y. P. Avalos, "Enhancing chemistry laboratory teaching through gamification: The IQ-Mobile Race," in Proc. 2023 IEEE Int. Conf. Teaching, Assessment and Learning for Engineering (TALE), 2023, pp. 1-4, doi: 10.1109/TALE56641.2023.10398328.
- [4] J. T. Bell and H. S. Fogler, "The application of virtual reality to (chemical engineering) education," in Proc. IEEE Virtual Reality Conf. (VR 2004), 2004, pp. 217-218, doi: 10.1109/VR.2004.1310077.
- [5] Y. Luo, P. Guo, S. Hasegawa, and M. Sato, "An interactive molecular visualization system for education in immersive multi-projection virtual environment," in Proc. 3rd Int. Conf. Image and Graphics (ICIG), 2004, pp. 485-488, doi: 10.1109/ICIG.2004.36.
- [6] T. Xie, Y. Li, and Y. Tang, "Effects of using immersive virtual reality for science education on learning outcomes: A randomized controlled pilot study," IEEE Trans. Learning Technologies, vol. 16, no. 6, pp. 1045-1056, 2023, doi: 10.1109/TLT.2023.3263587.
- [7] N. Sandu and E. Gide, "Adoption of AI-chatbots to enhance student learning experience in higher education in India," in Proc. 2019 18th Int. Conf. Information Technology Based Higher Education and Training (ITHET), 2019, pp. 1-5, doi: 10.1109/ITHET46829.2019.8937382.
- [8] M. P. Kurniawan, M. Suyanto, E. Utami, and Kusriani, "Educational game learning quality evaluation tool: A systematic literature review," in Proc. 2024 7th Int. Conf. Informatics and Computational Sciences (ICICoS), 2024, doi: 10.1109/ICICoS62600.2024.10636925.
- [9] J. K. Gilbert and D. F. Treagust, Eds., Multiple Representations in Chemical Education. Dordrecht, Netherlands: Springer, 2009, doi: 10.1007/978-1-4020-8872-8.
- [10] H.-K. Wu, J. S. Krajcik, and E. Soloway, "Using technology to support the development of conceptual understanding of chemical representations," 2000. [VERIFY: venue and pages]
- [11] E. Byusa, E. Kampire, and A. R. Mwesigye, "Game-based learning approach on students' motivation and understanding of chemistry concepts: A systematic review of literature," Heliyon, vol. 8, no. 5, e09541, 2022, doi: 10.1016/j.heliyon.2022.e09541.

- [12] N. Ali and S. Ullah, "Review to analyze and compare virtual chemistry laboratories for their use in education," *J. Chemical Education*, vol. 97, no. 10, pp. 3563-3574, 2020, doi: 10.1021/acs.jchemed.0c00185.
- [13] J. Alvarez Ramirez and A. Bueno, "Learning organic chemistry with virtual reality," in *Proc. 2020 IEEE Int. Conf. Engineering Veracruz (ICEV)*, 2020, doi: 10.1109/ICEV50249.2020.9289672.
- [14] I. Atmosukarto, W. S. Cheow, P. Iyer, N. Tong, and K. Yu, "Enhancing adaptive online chemistry course with AI-chatbot," in *Proc. 2021 IEEE Int. Conf. Engineering, Technology & Education (TALE)*, 2021, doi: 10.1109/TALE52509.2021.9678528.
- [15] Unity Technologies, "Unity 6 User Manual," 2025. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [16] The Khronos Group, "The OpenXR Specification," [Online]. Available: <https://registry.khronos.org/OpenXR/>
- [17] Unity Technologies, "XR Interaction Toolkit," package documentation. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@latest>
- [18] Convai, "Convai Documentation - Character API," [Online]. Available: <https://docs.convai.com/>

PUBLISHED PAPERS

[INSERT: Published / communicated paper "Atomix - Your Personal Lab with A.I."
by K. D. Soni, D. M. Thakar, K. R. Verma and M. Phadke]

[INSERT: Conference / journal name, date, ISSN/ISBN, DOI or acceptance letter]

CERTIFICATES

[INSERT: Paper publication certificates]

[INSERT: Conference attendance / presentation certificates]

[INSERT: Competition participation certificates, if any]

PLAGIARISM REPORT

[INSERT: First few pages of the plagiarism report (e.g., Turnitin / Drillbit) with similarity index]